

Haskell.py Teacher's Edition

A Complete Course in Idiomatic Functional Python

nine chapters, 41 homework problems, 10 compare-and-contrast apps

J. L. Clements III

September 18, 2026

How to Take This Course

This book teaches `haskell.py` – `prelude.py`’s production sibling. Where `prelude.py` reimplements every tool from scratch for pedagogy, `haskell.py` WRAPS the standard library wherever it already has the tool (`scanl` is `itertools.accumulate`, `memo` is `functools.cache`) – one vocabulary, zero reimplementation. Failure is a value throughout: a true NOTHING sentinel keeps `None` legal as everyday data, parsers fail with `FAIL`, and `Either` carries `WHY`.

The whole library is one listing, up front, in Chapter 1 – read it once, top to bottom, the way you would read any short source file. The nine chapters after it are not a second pass over the same material: each opens with why its slice of the library exists and how its pieces fit together, then goes straight to **Homework** – drills cement each tool, applies combine them, and challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in; `python3 -m doctest` is the referee.

Contents

How to Take This Course	iii
I The Nine Chapters	1
1 haskell.py, Complete	3
2 Point-Free Foundations	17
2.1 Homework	17
3 Folds, Scans and Unfolds	19
3.1 Homework	19
4 The List Gaps	21
4.1 Homework	21
5 Maybe, Done Right	23
5.1 Homework	23
6 Either: Failure With a Why	25
6.1 Homework	25
7 do-Notation	27
7.1 Homework	27
8 Parser Combinators	29
8.1 Homework	29
9 Monoids at Work	33
9.1 Homework	33
10 Capstone Labs	35
10.1 Homework	35
II Compare and Contrast: Imperative vs Functional	39
About this part	41
11 calculator	43
12 gradebook	45
13 layers	47

14	leaderboard	49
15	logtrriage	51
16	orderbatch	53
17	orgchart	55
18	payoff	57
19	sensoralert	59
20	wordfreq	61
III Appendices		63
A	haskell.py, Complete	65

Part I

The Nine Chapters

Chapter 1

haskell.py, Complete

Every name this book teaches, in the order it is defined, broken into the same seventeen sections `haskell.py` itself uses – each section’s code first, then commentary, a few verified examples, and for the sections whose mechanism most needs it, a step-by-step trace. Comments and docstrings are stripped from the listings themselves (that is what makes them worth calling “terse”); everything explanatory here is written fresh, not copied out of the source.

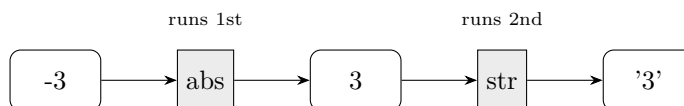
1. functions

```
from functools import reduce as _reduce, partial, cache as memo, cmp_to_key
from itertools import accumulate as _acc, islice as _islice, pairwise as _pairwise
from itertools import chain, count, cycle, repeat
from operator import add, sub, mul, not_, eq
from math import gcd, lcm, hypot, isqrt, prod as product
from collections import Counter, defaultdict, deque
from heapq import heappush, heappop, merge as _hmerge
import re as _re
identity = lambda x: x
const = lambda x: lambda _: x
flip = lambda f: lambda a,b: f(b,a)
curry = lambda f: lambda x: lambda y: f(x,y)
uncurry = lambda f: lambda p: f(*p)
def compose(*fs):
    def g(x):
        for f in reversed(fs):
            x = f(x)
        return x
    return g
o = compose
def pipe(x,*fs):
    for f in fs:
        x = f(x)
    return x
on = lambda f,g: lambda x,y: f(g(x),g(y))
```

This section is the adapter kit: none of these change what a value IS, only how it gets PASSED around. `compose` and `pipe` chain functions together (right-to-left and left-to-right respectively); `on` routes a comparison through a shared key first; `curry`/`uncurry` convert between a two-argument function and a function of one pair; `flip` swaps an argument order to fit a slot it wasn’t written for. The five one-liners at the top (`identity`, `const`, `flip`, `curry`, `uncurry`) are the smallest possible versions of these ideas – worth reading once, not worth a paragraph each.

```
>>> compose(str, abs)(-3)
'3'
>>> pipe(-3, abs, str)
'3'
>>> on(sub, len)('haskell', 'py')
5
>>> curry(add)(3)(4)
```

| 7

Illustration.

Even though `compose(str, abs)` is written left to right, `abs` runs FIRST – the diagram’s arrows show the actual data flow: `-3` into `abs` gives `3`, and only then into `str` gives `'3'`.

2. pairs

```
fst = lambda p: p[0]
snd = lambda p: p[1]
swap = o(tuple, reversed)
```

Pair accessors. `fst/snd` read a 2-tuple’s first or second element by name instead of by index, which reads better inside a `sortOn` key or a dict comprehension than a bare `[0]/[1]` does. `swap` exchanges the two – built from `o(tuple, reversed)`, composition rather than indexing.

```
>>> fst(('a', 1))
'a'
>>> snd(('a', 1))
1
>>> swap((1, 2))
(2, 1)
```

3. sentinels

```
class _Sentinel:
    __slots__ = ("_n",)
    def __init__(self, n): self._n = n
    def __repr__(self): return self._n
    def __bool__(self): return False
NOTHING = _Sentinel("NOTHING")
FAIL = _Sentinel("FAIL")
_MISS = _Sentinel("_MISS")
```

Three sentinel OBJECTS, not values – each is a distinct `_Sentinel` instance that is never equal to anything except itself, so none of them can be confused with a legitimate piece of data. `NOTHING` means “no answer” (Maybe’s absence, distinct from a stored `None`); `FAIL` means “the parser gave up”; `_MISS` is a private default meaning “no argument was supplied”, used internally by `foldl/scanl` so a real `None` can still be passed as a seed. `isJust/isNothing` name the `NOTHING` test itself as a predicate, so it can be handed to `find`, `filter_`, or `all` directly.

```
>>> isJust(5)
True
>>> isNothing(NOTHING)
True
>>> NOTHING == None
False
```

4. arithmetic & logic

```
div = lambda a,b: a // b
mod = lambda a,b: a % b
halve = lambda n: n // 2
even = lambda n: n % 2 == 0
odd = lambda n: n % 2 == 1
```

```

otherwise = True
quot = lambda a,b: -(-a // b) if (a < 0) != (b < 0) else a // b
rem = lambda a,b: a - b * quot(a,b)
quotRem = lambda a,b: (quot(a,b), rem(a,b))
signum = lambda x: (x > 0) - (x < 0)
succ = lambda x: chr(ord(x) + 1) if isinstance(x,str) else x + 1
pred = lambda x: chr(ord(x) - 1) if isinstance(x,str) else x - 1

```

Small tools with a sharp edge: Python's own `//` and `%` FLOOR toward negative infinity, but Haskell's `quot/rem` TRUNCATE toward zero – the two families disagree the moment a negative number is involved, and this section keeps both available under their own names rather than picking one and hiding the difference. `signum` compresses a three-way comparison into arithmetic; `succ/pred` work on both numbers and single characters.

```

>>> quot(-7, 2)
-3
>>> rem(-7, 2)
-1
>>> -7 // 2, -7 % 2
(-4, 1)
>>> signum(-5)
-1
>>> succ('a')
'b'

```

5. folds, scans, unfolds

```

foldl = lambda f,xs,base=_MISS: _reduce(f,xs) if base is _MISS else _reduce(f,xs,base)
foldr = lambda f,xs,base: _reduce(flip(f),reversed(list(xs)),base)
foldl1 = foldl
foldr1 = lambda f,xs: _reduce(flip(f),reversed(list(xs)))
scanl = lambda f,xs,base=_MISS: list(_acc(xs,f) if base is _MISS else _acc(xs,f,initial=base))
scanl1 = scanl
scanr = lambda f,xs,base: scanl(flip(f),reversed(list(xs)),base)[::-1]
scanr1 = lambda f,xs: scanl(flip(f),reversed(list(xs)))[::-1]
def unfoldr(f,seed):
    out = []
    while (r := f(seed)) is not NOTHING:
        out.append(r[0]); seed = r[1]
    return out
def until(cond,f,x):
    while not cond(x):
        x = f(x)
    return x
replicateM = lambda n,xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]])
subsequences = lambda xs: foldl(lambda acc,x: acc + [s + [x] for s in acc], list(xs), [[]])

```

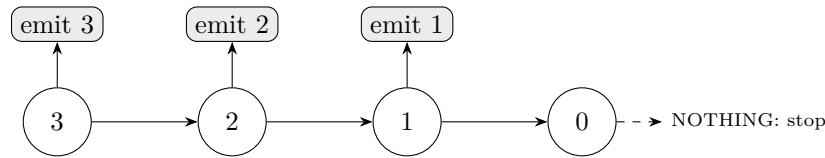
The trunk of the file. A FOLD collapses a list to one value; a SCAN is a fold that keeps every intermediate accumulator instead of just the last; an UNFOLD runs the whole idea backwards, growing a list from a seed instead of consuming one. `scanl` here is exactly `itertools.accumulate` under its Haskell name – naming it differently doesn't change what runs. `unfoldr`'s step function returns (value, next_seed) to keep going, or NOTHING to stop; that single convention is why countdown timers, digit sequences, and Collatz chains all turn out to be one `unfoldr` call apiece.

```

>>> foldl(lambda a, b: a - b, [1, 2, 3], 10)
4
>>> scanl(add, [1, 2, 3], 0)
[0, 1, 3, 6]
>>> unfoldr(lambda n: NOTHING if n == 0 else (n, n - 1), 3)
[3, 2, 1]

```

Illustration.



Each circle is the SEED at that step; each step both emits a value (up) and produces the next seed (right), until a seed maps to NOTHING instead of a (value, next_seed) pair, which is where the whole unfold stops – seed 0 never gets its own emitted value.

6. streams

```
def iterate(f,x):
    while True:
        yield x
        x = f(x)
take = lambda n,xs: list(_islice(iter(xs),n))
```

Two names, one idea: a stream can be INFINITE because nothing is computed until something asks for it. `iterate(f, x)` yields `x`, `f(x)`, `f(f(x))`, ... forever; `take(n, ...)` is what makes an infinite generator usable by cutting it off after `n` elements – `chain`, `count`, `cycle`, and `repeat` (imported straight from `itertools` in section 1's import block) round out the family.

```
>>> take(5, iterate(lambda n: n * 2, 1))
[1, 2, 4, 8, 16]
```

7. list basics

```
head = lambda xs: xs[0]
tail = lambda xs: xs[1:]
init = lambda xs: xs[:-1]
last = lambda xs: xs[-1]
drop = lambda n,xs: list(xs)[max(n,0):]
splitAt = lambda n,xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs)
replicate = lambda n,x: [x] * n
elem = lambda x,xs: x in xs
notElem = lambda x,xs: x not in xs
null = lambda xs: len(xs) == 0
nub = o(list,dict.fromkeys)
enum = lambda xs,start=0: list(enumerate(xs,start))
pairwise = lambda xs: list(_pairwise(xs))
zip_ = lambda a,b: list(zip(a,b))
zip3 = lambda a,b,c: list(zip(a,b,c))
unzip = lambda ps: tuple(map(list,zip(*ps))) if ps else ([], [])
isPrefixOf = lambda p,xs: list(xs[:len(p)]) == list(p)
isSuffixOf = lambda p,xs: list(xs[len(xs) - len(p):]) == list(p)
lookup = lambda k,pairs: next((v for kk, v in pairs if kk == k),NOTHING)
stripPrefix = lambda p,xs: xs[len(p):] if isPrefixOf(p,xs) else NOTHING
```

The list's own anatomy: taking it apart at the ends (`head/tail/init/ last`), cutting it at a position (`drop/splitAt`), and pairing it against itself (`pairwise` zips a list against its own tail, which is why every rule about ADJACENT elements starts there). `lookup` and `stripPrefix` both return NOTHING rather than raising or returning an ambiguous `None` on failure.

```
>>> pairwise([1, 2, 3])
[(1, 2), (2, 3)]
>>> stripPrefix('foo', 'foobar')
'bar'
>>> stripPrefix('x', 'foobar') is NOTHING
True
```

8. higher-order lists

```

map_ = lambda f,xs: [f(x) for x in xs]
filter_ = lambda cond,xs: [x for x in xs if cond(x)]
concat = lambda xss: [x for xs in xss for x in xs]
concatMap = lambda f,xs: [y for x in xs for y in f(x)]
cross = lambda a,b: [(x, y) for x in a for y in b]
starmap = lambda f,pairs: [f(*p) for p in pairs]
zipWith = lambda f,a,b: [f(x,y) for x, y in zip(a,b)]
zipWith3 = lambda f,a,b,c: [f(x,y,z) for x, y, z in zip(a,b,c)]
find = lambda cond,xs: next((x for x in xs if cond(x)),NOTHING)
def partition(cond,xs):
    yes, no = [], []
    for x in xs:
        (yes if cond(x) else no).append(x)
    return (yes, no)

```

Where loops go to be named. `map_` transforms, `filter_` selects, `concatMap` enumerates (map, then flatten, in one pass), `zipWith` combines two lists element-wise. Two tools do what a raw loop does badly: `find` stops at the first match even on an infinite stream, and `partition` splits a list into (keepers, rest) in a single pass with the predicate called exactly once per element.

```

>>> concatMap(lambda x: [x, x], [1, 2])
[1, 1, 2, 2]
>>> find(even, [1, 3, 4, 5])
4
>>> partition(even, [1, 2, 3, 4])
([2, 4], [1, 3])

```

9. slicing & spans

```

def span(cond,xs):
    xs = list(xs)
    i = next((i for i, x in enumerate(xs) if not cond(x)),len(xs))
    return (xs[:i], xs[i:])
break_ = lambda cond,xs: span(o(not_,cond),xs)
def takewhile(cond,xs):
    for x in xs:
        if not cond(x): return
    yield x
def dropwhile(cond,xs):
    it = iter(xs)
    for x in it:
        if not cond(x):
            yield x
            break
    yield from it
def takeuntil(cond,xs):
    for x in xs:
        yield x
        if cond(x): return
takeWhile = lambda cond,xs: list(takewhile(cond,xs))
dropWhile = lambda cond,xs: list(dropwhile(cond,xs))
inits = lambda xs: [xs[:i] for i in range(len(xs) + 1)]
tails = lambda xs: [xs[i:] for i in range(len(xs) + 1)]
def chunksOf(n,xs):
    xs = list(xs)
    return [xs[i:i + n] for i in range(0,len(xs),n)]
def windows(n,xs):
    xs = list(xs)
    return [xs[i:i + n] for i in range(len(xs) - n + 1)]
def groupBy(eq,xs):
    out = []
    for x in xs:
        if out and eq(out[-1][0],x): out[-1].append(x)

```

```

        else: out.append([x])
    return out
group = partial(groupBy,eq)
def transpose(rows):
    rows = [list(r) for r in rows]
    out, i = [], 0
    while any(i < len(r) for r in rows):
        out.append([r[i] for r in rows if i < len(r)])
        i += 1
    return out

```

Finite VIEWS of a sequence. `span/break_` cut a sequence at the first place a condition changes – a lexer in one call. `windows(n, xs)` produces every contiguous length-`n` slice (a genuine sliding view, not index arithmetic), while `groupBy` segments a sequence into runs of CONSECUTIVE equal elements, each new element compared against its run’s first member. `chunksOf` pages anything into fixed-size (possibly ragged-last) pieces.

```

>>> windows(3, [1, 2, 3, 4])
[[1, 2, 3], [2, 3, 4]]
>>> groupBy(lambda a, b: a == b, 'aabba')
[['a', 'a'], ['b', 'b'], ['a']]
>>> chunksOf(2, [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]

```

10. sorting & searching

```

sortOn = lambda f,xs: sorted(xs,key=f)
sortBy = lambda cmp,xs: sorted(xs,key=cmp_to_key(cmp))
maxOn = lambda f,xs: max(xs,key=f)
minOn = lambda f,xs: min(xs,key=f)
merge = lambda a,b: list(_hmerge(a,b))

```

“Does order unlock it?” is the first question worth asking of almost any problem: sorting linearises it, and this section is the toolkit for once it has. `sortOn` sorts by a computed key (the key function runs once per element, not once per comparison); `minOn`/`maxOn` answer “best by key” in one linear pass instead of sorting the whole list just to look at one end.

```

>>> sortOn(len, ['abc', 'a', 'ab'])
['a', 'ab', 'abc']
>>> maxOn(len, ['a', 'abc', 'ab'])
'abc'

```

11. strings

```

lines = str.splitlines
unlines = lambda ls: "".join(l + "\n" for l in ls)
words = str.split
unwords = " ".join

```

Four names turning text into lists and back: `words/unwords` split and rejoin on whitespace, `lines/unlines` split and rejoin on newlines. Once text becomes a list, every list tool in this file applies to it – that conversion is usually the first step, not an afterthought.

```

>>> words('a b c')
['a', 'b', 'c']
>>> unwords(['a', 'b', 'c'])
'a b c'

```

12. containers

```

def fromListWith(f,pairs):

```

```

d = {}
for k, v in pairs:
    d[k] = f(v,d[k]) if k in d else v
return d
insertWith = lambda f,k,v,d: {**d, k: f(v,d[k]) if k in d else v}
unionWith = lambda f,a,b: {k: f(a[k],b[k]) if k in a and k in b else a[k] if k in a else b[k]
                           for k in a.keys() | b.keys()}
def getpath(t,ks,default=NOTHING):
    for k in ks:
        if not isinstance(t,dict) or k not in t: return default
        t = t[k]
    return t
bag_diff = lambda a,b: {k: a[k] - b.get(k,0) for k in a if a[k] > b.get(k,0)}
bag_inter = lambda a,b: {k: v for k in a.keys() & b.keys() if (v := min(a[k],b[k]))}
bag_union = lambda a,b: {k: max(a.get(k,0),b.get(k,0)) for k in a.keys() | b.keys()}
bag_sub = lambda a,b: all(b.get(k,0) >= n for k,n in a.items())

```

Aggregation by key – a fold aimed at a dictionary instead of a list. `fromListWith` is THE dict-building fold: given (key, value) pairs and a combiner, it builds one dict, calling the combiner as `f(new, old)` on a collision. `unionWith` does the same job for two EXISTING dicts. Choosing the combiner is choosing what the merge MEANS: addition counts, list-append groups, `max` takes a bag union.

Analogy. Think of `fromListWith` as a tally sheet at a polling station: each ballot is a (candidate, 1) pair, and every ballot either opens a new tally or adds to an existing one using whatever rule you hand it (here, addition). `unionWith` is the same idea for MERGING two already-tallied sheets from two polling stations – the combiner says how to reconcile a candidate who appears on both.

`getpath` deserves its own picture: a nested dict IS a tree, and `getpath(t, ['a', 'b', 'c'])` walks it exactly the way you'd cd `a/b/c` on a filesystem – one directory at a time, and the moment a directory doesn't exist, you stop and report “not found” (NOTHING) instead of creating it. That last part matters: `getpath` only ever READS the tree, it never grows one by accident the way indexing a `defaultdict` silently would.

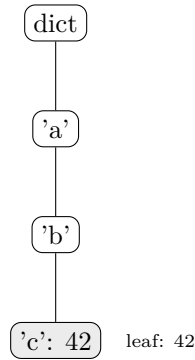
Bags (multisets). A `Counter`-style dict of {item: count} is a BAG – like a bag of groceries where you've written how many of each fruit you're carrying, not just which fruits you have. The four `bag_*` functions are the four things you'd naturally do with two such bags. `bag_union` restocks a shelf from two deliveries, keeping whichever delivery brought MORE of each fruit (`max` per item, not added-together totals – it's a union of what's available, not a combined order). `bag_inter` asks “what could I make using only fruit BOTH bags have”, so it keeps the SMALLER count per item, and drops any fruit that isn't in both bags at all. `bag_diff` is “what's left in bag a after bag b eats its share” – subtraction per item, but clipped so a fruit never goes negative or appears with a zero-or-less count. `bag_sub` answers a yes/no question: could bag a have been SCOOPED OUT of bag b – is there enough of every fruit in b to cover what a claims to have? If even one fruit in a exceeds what b has, the answer is `False`.

```

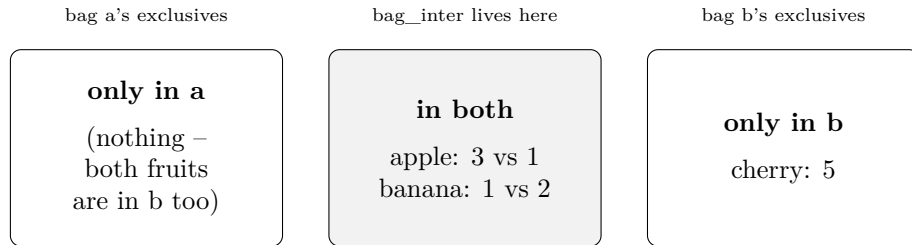
>>> fromListWith(add, [('a', 1), ('a', 2), ('b', 3)])
{'a': 3, 'b': 3}
>>> unionWith(add, {'a': 1}, {'a': 2, 'b': 3})
{'a': 3, 'b': 3}
>>> getpath({'a': {'b': {'c': 42}}}, ['a', 'b', 'c'])
42
>>> getpath({'a': {}}, ['a', 'b', 'c']) is NOTHING
True
>>> a = {'apple': 3, 'banana': 1}
>>> b = {'apple': 1, 'banana': 2, 'cherry': 5}
>>> bag_union(a, b)
{'apple': 3, 'banana': 2, 'cherry': 5}
>>> bag_inter(a, b)
{'apple': 1, 'banana': 1}
>>> bag_diff(a, b)
{'apple': 2}
>>> bag_sub(a, b)
False

```

Illustration.



`getpath` descends this tree exactly the way `cd a/b/c` descends a filesystem, one level per key – the moment a key is missing at any level, it stops right there and reports **NOTHING** instead of continuing or creating the missing branch.



For `a = {'apple': 3, 'banana': 1}` and `b = {'apple': 1, 'banana': 2, 'cherry': 5}`: `bag_union` takes the LARGER count per fruit wherever it appears (apple 3, banana 2, cherry 5); `bag_inter` takes the SMALLER count but only from the middle column (apple 1, banana 1 – cherry is dropped, it's not in both); `bag_diff(a, b)` keeps only what's left of `a` after subtracting `b`'s share, clipped at zero (apple $3 - 1 = 2$; banana $1 - 2 = -1$ drops out); and `bag_sub(a, b)` asks whether `a` could be scooped entirely out of `b` – it can't, because `b` only has 1 apple and `a` claims 3.

13. Maybe

```

bind = lambda x,f: NOTHING if x is NOTHING else f(x)
isJust = lambda x: x is not NOTHING
isNothing = lambda x: x is NOTHING
fromMaybe = lambda default,x: default if x is NOTHING else x
listToMaybe = lambda xs: xs[0] if xs else NOTHING
maybeToList = lambda x: [] if x is NOTHING else [x]
catMaybes = lambda xs: [x for x in xs if x is not NOTHING]
mapMaybe = lambda f,xs: [y for y in map(f,xs) if y is not NOTHING]
def sequenceM(xs):
    out = []
    for x in xs:
        if x is NOTHING: return NOTHING
        out.append(x)
    return out
traverseM = lambda f,xs: sequenceM(map_(f,xs))
maybe_get = lambda d,k: d.get(k,NOTHING)

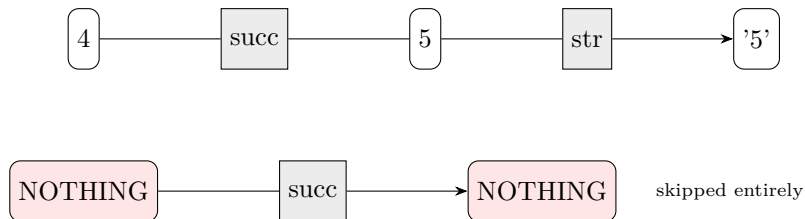
```

The Maybe monad. `bind` is Haskell's `>=` for this file's Maybe: it passes a value on to the next step unless that value is already **NOTHING**, in which case it short-circuits without calling anything. `sequenceM`/`traverseM` extend that idea to a whole list – ALL of it succeeds, or the combined result is **NOTHING** – and `mapMaybe`/`catMaybes` are the “map into Maybe, then keep only the successes” pair.

Analogy. `bind` is a factory conveyor belt with a quality inspector stationed at every workstation. Each inspector checks the part that just arrived; if it's already broken (**NOTHING**), they wave the whole belt to a stop right there – no downstream station ever even LOOKS at a broken part, let alone tries to work on it. A Maybe VALUE is a part on the belt (or the “broken” signal itself); `bind` is one inspector's station. Chaining several binds is the whole assembly line: one break anywhere, and the finished product at the end is automatically **NOTHING**, with no station having to explicitly ask “did the last guy fail?” `sequenceM` is the same idea applied to a whole PALLET of parts at once: the pallet only ships if every single part on it passed inspection.

```
>>> bind(4, succ)
5
>>> bind(NOTHING, succ) is NOTHING
True
>>> sequenceM([1, 2, 3])
[1, 2, 3]
>>> sequenceM([1, NOTHING, 3]) is NOTHING
True
```

Illustration.



Top row: a healthy chain, `bind(bind(4, succ), str)`, each inspector finding real work to do. Bottom row: had the FIRST value already been `NOTHING`, the `succ` inspector would find nothing to inspect and just wave `NOTHING` through untouched – it never even tries to call `succ` on it.

14. Either

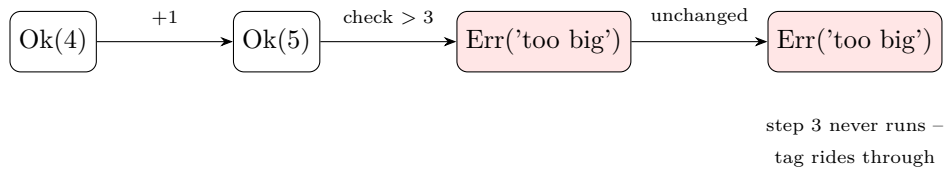
```
Ok = lambda v: ("ok", v)
Err = lambda e: ("err", e)
bindE = lambda x,f: x if x[0] == 'err' else f(x[1])
def sequenceE(xs):
    out = []
    for x in xs:
        if x[0] == "err": return x
        out.append(x[1])
    return Ok(out)
traverseE = lambda f,xs: sequenceE(map_(f,xs))
def partitionEithers(xs):
    oks, errs = [], []
    for tag, v in xs:
        (oks if tag == "ok" else errs).append(v)
    return (oks, errs)
note = lambda msg,x: Err(msg) if x is NOTHING else Ok(x)
```

Where `Maybe` only says “it failed”, `Either` says WHY. `Ok/Err` are tagged pairs – (“ok”, value) or (“err”, reason) – and `bindE` short-circuits on the first `Err` exactly like `bind` does on `NOTHING`, but keeps the message riding along. `note` is the bridge FROM `Maybe`: it turns a `NOTHING` into an `Err` carrying a reason you supply.

Analogy. If `Maybe` is a doctor’s yes/no answer to “are you okay?”, `Either` is the doctor’s note that also names the symptom: not just “no”, but “no, your temperature is 102”. The factory line from section 13 gets the same upgrade – each inspector, instead of just halting the belt on a broken part, ATTACHES A TAG explaining what was wrong with it, and that tag rides along, unread by any later station, until it reaches whoever is waiting at the end of the line to read the final report. `bindE` is that tag-preserving inspector; `sequenceE` is the whole-pallet version, reporting the FIRST tag it finds rather than just “something on this pallet was bad”.

```
>>> Ok(5)
('ok', 5)
>>> Err('bad input')
('err', 'bad input')
>>> bindE(Ok(4), lambda v: Ok(v + 1))
('ok', 5)
>>> bindE(Err('boom'), lambda v: Ok(v + 1))
('err', 'boom')
```

Illustration.



The tag `'too big'` is attached at step 2 (the only place that knows WHY things went wrong), then simply carried along by every `bindE` after it, unread and unchanged, until whoever is waiting at the end finally reads it – step 3’s doubling logic never even runs.

15. do-notation

```

def do(binder, is_fail):
    def deco(gen_fn):
        def run(*a, **kw):
            g = gen_fn(*a, **kw)
            def step(val):
                try:
                    m = g.send(val)
                except StopIteration as e:
                    return e.value
                return binder(m, step)
            return step(None)
        return run
    return deco
doM = do(bind, lambda x: x is NOTHING)
doE = do(bindE, lambda x: x[0] == "err")

```

Chaining `bind` calls by hand nests one call inside the next, one level of indentation per step – the “bind pyramid”. `do` rebuilds that same chain from an ordinary generator function instead: each `yield` sends a `Maybe` (or `Either`) value to `do`’s machinery, which unwraps it and sends the unwrapped value back as the `yield` expression’s result, or aborts the whole function the moment any step fails. `doM` wires this to `Maybe`, `doE` to `Either` – same generator shape, different short-circuit rule.

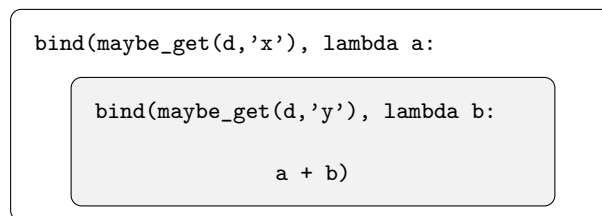
Analogy. A generator written with `@doM` reads like a recipe written in plain imperative steps – “get the flour, get the sugar, mix them” – even though every single step could secretly fail. `do` is the kitchen assistant standing behind you: you write the recipe as if nothing ever goes wrong, and the assistant is the one who actually checks the pantry before handing you each ingredient, silently walking away with the whole recipe abandoned the moment one ingredient is missing. You never write the “is it there?” check yourself – `yield` IS that check, happening invisibly at every line.

```

>>> @doM
... def h(d):
...     a = yield maybe_get(d, 'x')
...     b = yield maybe_get(d, 'y')
...     return a + b
>>> h({'x': 1, 'y': 2})
3
>>> h({'x': 1}) is NOTHING
True

```

Illustration.



This is what `do` builds from the generator above – each `yield` is one more box NESTED inside the last, exactly the “bind pyramid” from indented, hand-written binds. The generator syntax just lets you write the two lines flat, top to bottom, while `do` does the nesting for you behind the scenes. Calling `h({'x': 1})` is like the kitchen assistant reaching for sugar that was never bought: the OUTER box’s check fails, and the inner box (the mixing step) is never even reached.

16. parser combinators

```

def doP(gen_fn):
    def make(*a,**kw):
        def parser(s):
            g, val = gen_fn(*a,**kw), None
            try:
                while True:
                    r = g.send(val)(s)
                    if r is FAIL: return FAIL
                    val, s = r
            except StopIteration as e:
                return (e.value, s)
        return parser
    return make
alt = lambda *ps: lambda s: next((r for r in (p(s) for p in ps) if r is not FAIL),FAIL)
def many(p):
    def parser(s):
        out = []
        while (r := p(s)) is not FAIL and r[1] != s:
            out.append(r[0]); s = r[1]
        return (out, s)
    return parser
def many1(p):
    def parser(s):
        r = p(s)
        if r is FAIL: return FAIL
        rest = many(p)(r[1])
        return ([r[0]] + rest[0], rest[1])
    return parser
def sepBy(p,sep):
    def pair(s):
        r = sep(s)
        return FAIL if r is FAIL else p(r[1])
    def parser(s):
        r = p(s)
        if r is FAIL: return ([], s)
        rest = many(pair)(r[1])
        return ([r[0]] + rest[0], rest[1])
    return parser
def lit(c):
    def parser(s):
        s = s.lstrip()
        return (c, s[1:]) if s[:1] == c else FAIL
    return parser
def rx(pat,conv=identity):
    r = _re.compile(pat)
    def parser(s):
        s = s.lstrip()
        m = r.match(s)
        return (conv(m.group()), s[m.end():]) if m else FAIL
    return parser
def chain1(p,ops):
    opp = alt(*[lit(c) for c in ops])
    def parser(s):
        vr = p(s)
        while vr is not FAIL:
            op = opp(vr[1])
            if op is FAIL: return vr
            w = p(op[1])
            if w is FAIL: return vr
            vr = (ops[op[0]](vr[0],w[0]), w[1])
        return FAIL

```

```

    return parser
def runParser(p,s):
    r = p(s)
    if r is FAIL: return Err(f"no parse: {s!r}")
    if r[1].strip(): return Err(f"unconsumed input: {r[1].strip()!r}")
    return Ok(r[0])

```

A parser here is just a function from a string to (value, rest) or FAIL – once that shape is fixed, parsers compose like any other function. `doP` threads the remaining input through a generator the same way `doM` threads a Maybe; `alt` tries alternatives in order; `many/sepBy` handle repetition and separator-delimited lists without recursion (so they cost no stack, no matter how long the input). `chain11` folds a sequence of `p (op p)*` strictly LEFT, which is how ordinary arithmetic is supposed to associate.

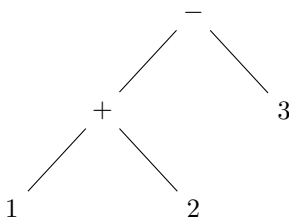
Analogy. Parsing a string is like eating a plate of food one bite at a time, always reporting how much plate is left after each bite: a parser is one BITE (it consumes some prefix of the input) plus a report of the leftovers. `alt` is “try dish A; if you can’t stomach it, try dish B instead, from the same starting plate”. `many` is “keep taking bites of the same dish until there’s nothing left you can eat”. `doP` lets you describe a whole MEAL as a sequence of bites, one `yield` per course, with the plate (the remaining input) silently passed from bite to bite behind the scenes – you never carry the plate yourself.

```

>>> n = rx(r'~?\d+', int)
>>> runParser(sepBy(n, lit(',')), '1,2,3')
('ok', [1, 2, 3])
>>> runParser(chain11(n, {'+': add, '-': sub}), '1+2-3')
('ok', 0)

```

Illustration.



`chain11` builds this tree LEFT-heavy, one operator at a time: 1 and 2 fold together under + FIRST (the tree’s left-hand branch), and only then does that whole result fold with 3 under – – exactly $(1 + 2) - 3$, never $1 + (2 - 3)$. This is the same shape as a running restaurant tab: each new item is added to (or discounted from) the running total so far, left to right, and once tallied it is never revisited.

17. monoids

```

Monoid = lambda empty,op: (empty, op)
mconcat = lambda m,xs: foldl(m[1],xs,m[0])
foldMap = lambda f,m,xs: mconcat(m,[f(x) for x in xs])
both = lambda m1,m2: Monoid((m1[0], m2[0]),
    lambda a,b: (m1[1](a[0],b[0]), m2[1](a[1],b[1])))
Sum = Monoid(0,add)
Product = Monoid(1,mul)
All = Monoid(True,lambda a,b: a and b)
Any = Monoid(False,lambda a,b: a or b)
MaxM = Monoid(float("-inf"),max)
MinM = Monoid(float("inf"),min)
ListM = Monoid([],add)
First = Monoid(NOTHING,lambda a,b: b if a is NOTHING else a)
Last = Monoid(NOTHING,lambda a,b: a if b is NOTHING else b)

```

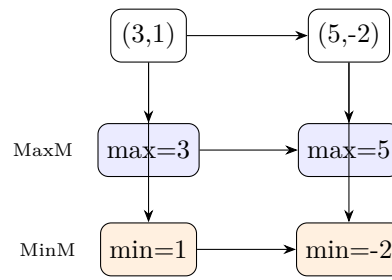
A monoid is nothing but an identity element paired with an associative combiner, reified as the pair (empty, op) – naming it as DATA means one engine, `mconcat`, folds every instance, and `foldMap` fuses “measure each element” with “combine the measurements” into one call. `both` is the payoff: it pairs two monoids into one, so two statistics (a max AND a min, a count AND a total) fall out of a SINGLE traversal instead of two separate loops.

Analogy. A monoid is a “combine two of these into one” rule that comes bundled with its own honest “empty” starting point – the way a shopping cart’s running total combines with a starting balance of \$0 (not \$1, not “undefined”), and adding zero items

never breaks the rule. **Sum** is that shopping-cart total; **MaxM** is a scoreboard that starts at negative infinity so the very first score posted is guaranteed to beat it; **ListM** is string concatenation's starting point, the empty string, generalised to any list. **both** is running TWO REGISTERS over the same single pass of items through a checkout line – one register tallying cost, the other counting items – instead of scanning the cart twice, once per register.

```
>>> mconcat(Sum, [1, 2, 3])
6
>>> foldMap(len, Sum, ['ab', 'c'])
3
>>> both(MaxM, MinM)[1]((3, 1), (5, -2))
(5, -2)
```

Illustration.



Both registers read the SAME incoming pair at each step – there's only one pass through the data, top row – and each runs its own independent fold, MaxM tracking the running maximum and MinM the running minimum. **both(MaxM, MinM)[1]** is the combined operator that updates both registers at once; by the second item, the running answer is already (5, -2), a max AND a min from one traversal, not two separate scans over the data.

Chapter 2

Point-Free Foundations

A function that names its argument commits to one shape of input; a function built from other functions (`compose`, `on`, `partial`) stays a transformation, reusable wherever the types line up. This chapter’s five one-liners (`identity`, `const`, `flip`, `curry`, `uncurry`) are the adapters that make composition possible at all – they change nothing about the values, only the SHAPE of the call. Two sentinels close the chapter: `NOTHING` means “no answer”, and `FAIL` means “the parser gave up” – both real objects, never `None`, so a function can return an ordinary `None` as genuine data without it being mistaken for absence.

2.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest` referee.

2.1 Mirror a pair

drill

Write `mirror(p)`: the pair with its elements exchanged, point-free. Applying it twice must give the original back.

`mirror(p) -> pair`

```
>>> mirror((1, 2))
(2, 1)
>>> mirror(mirror((3, 4)))
(3, 4)
```

Solution.

```
mirror = o(tuple, reversed)
```

Note. `mirror = o(tuple, reversed)` is composition, read right to left: `reversed(p)` yields the pair’s two elements back to front (like an iterator over `(p[1], p[0])`), and `tuple(...)` then materializes that into an actual tuple. Because the whole thing is built from function composition, no argument is ever named or indexed – there is no `p[0]/p[1]` anywhere in the definition.

2.2 Record projections

drill

A leaderboard record is a `(name, score)` pair. Write `name_of` and `score_of` point-free, without lambdas.

`name_of(r) -> value; score_of(r) -> value`

```
>>> name_of(('ada', 95))
'ada'
>>> score_of(('ada', 95))
95
```

Solution.

```
name_of, score_of = itemgetter(0), itemgetter(1)
```

Note. `operator.itemgetter(0)` and `itemgetter(1)` build extractor functions without writing `lambda r: r[0]` by hand – `itemgetter` is exactly Haskell’s `fst/snd` generalized to any index. Because `itemgetter` returns a genuine function object rather than a lambda, it slots directly into `map(name_of, records)` or `sorted(records, key=score_of)` with no wrapping needed.

2.3 Default when missing

drill

Write `port_of(cfg)`: the 'port' entry of a config dict, defaulting to 8080 only when the key is absent. A key holding `None` is real data and must survive.

`port_of(cfg) -> value`

```
>>> port_of({'port': 9000})
9000
>>> port_of({})
8080
>>> port_of({'port': None}) is None
True
```

Solution.

```
| port_of = lambda cfg: fromMaybe(8080, maybe_get(cfg, 'port'))
```

Note. `maybe_get(cfg, 'port')` looks up the key and returns the NOTHING sentinel specifically when the key is absent, never confusing that with a key that is present but stored as `None`. `fromMaybe(8080, ...)` only substitutes its default in the NOTHING case, so an explicitly stored `None` passes through untouched – which is exactly why the third test (port set to `None`) must come back as `None`, not 8080.

2.4 Clamp to a range

applied

Write `clamp(lo, hi)`: a FUNCTION that forces a number into `[lo, hi]`. Build it from `max` and `min` with partial application – no `lambda` in the returned function.

`clamp(lo, hi) -> (x -> value)`

```
>>> clamp(0, 10)(15)
10
>>> clamp(0, 10)(-5)
0
>>> clamp(0, 10)(7)
7
```

Solution.

```
| clamp = lambda lo, hi: o(partial(max, lo), partial(min, hi))
```

Note. `partial(max, lo)` and `partial(min, hi)` each freeze one argument, turning `max/min` into one-argument functions (Haskell-style sections). Composing them right to left with `o` means `min(hi, x)` runs FIRST, capping the value from above, and its result feeds into `max(lo, ...)`, which floors it from below – both bounds enforced in one pipeline, no `if/elif` needed.

2.5 Twice

drill

Write `twice(f)`: the function that applies `f` two times.

`twice(f) -> (x -> value)`

```
>>> twice(lambda n: n * 3)(2)
18
>>> twice(str.strip)(' hi ')
'hi'
```

Solution.

```
| twice = lambda f: o(f, f)
```

Note. `o(f, f)` composes `f` with itself, so `twice(f)(x)` equals `f(f(x))` by definition of composition. Because `o` is an ordinary function rather than special syntax, nothing stops it from taking the SAME function twice, and applying twice a second time – `twice(twice(f))` – runs `f` four times, since composition itself composes.

Chapter 3

Folds, Scans and Unfolds

A fold collapses a list to one value; a scan is a fold that keeps its history; an unfold grows a list from a seed instead of consuming one. `scanl` here is *exactly* `itertools.accumulate` – naming it the Haskell way does not change what runs. `unfoldr` is the fold’s mirror image: given a step function that returns `(value, next_seed)` or `NOTHING` to stop, it produces the whole list, which is why balances, digit sequences and Collatz chains all turn out to be one `unfoldr` call apiece. `iterate` is `unfoldr`’s lazy, infinite cousin: it never stops on its own, so `take` is what makes it usable.

3.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

3.1 Parity census

drill

Count evens and odds as a pair (evens, odds), in ONE pass, point-free at the top level.

```
parity_counts(xs) -> (evens, odds)
```

```
>>> parity_counts([1, 2, 3, 4])
(2, 2)
>>> parity_counts([])
(0, 0)
>>> parity_counts([-2, -3])
(1, 1)
```

Solution.

```
| parity_counts = partial(foldMap, lambda x: (1 - x % 2, x % 2), both(Sum, Sum))
```

Note. `foldMap` first measures each element with `lambda x: (1 - x % 2, x % 2)`, turning every value into a (is_even, is_odd) pair of 0s and 1s; `both(Sum, Sum)` then runs two independent `Sum` folds in lockstep, one over each tuple position. `partial` freezes the measuring function and the monoid together into a single reusable name, and because `Sum`’s identity is 0, the empty-list case `((0, 0))` falls out for free with no special-casing.

3.2 Running balance

drill

An account starts at 0; deposits are positive, withdrawals negative. Return every intermediate balance including the start.

```
balances(txs) -> [balance]
```

```
>>> balances([100, -30, -30, 5])
[0, 100, 70, 40, 45]
>>> balances([])
[0]
```

Solution.

```
| balances = lambda txs: scanl(add, txs, 0)
```

Note. `scanl(add, txs, 0)` is a fold that keeps EVERY intermediate accumulator instead of just the final one: starting from the seed 0, it emits the running total after each transaction in `txs` in turn, so the result is one element longer than the input (it includes the starting balance before any transaction is applied). `operator.add` stands in for `lambda a, b: a + b`, following the file’s point-free convention.

3.3 Countdown

drill

Write `countdown(n)`: the list `n, n-1, ..., 1` grown by `unfoldr`, point-free at the top level.

`countdown(n) -> [int]`

```
>>> countdown(4)
[4, 3, 2, 1]
>>> countdown(0)
[]
```

Solution.

```
countdown = partial(unfoldr, lambda n: NOTHING if n == 0 else (n, n - 1))
```

Note. `unfoldr` grows a list from a seed by repeatedly calling its step function and collecting what it returns. Here the step returns `(n, n - 1)` – this element, next seed – for every `n` except 0, and returns `NOTHING` once `n` hits 0, which `unfoldr` treats as the stop signal. `partial` freezes the step function in, so `countdown` ends up a plain function of `n` alone.

3.4 Digits of a number

applied

Write `digits(n)`: the base-10 digits of a non-negative integer, most significant first. Zero has one digit.

`digits(n) -> [int]`

```
>>> digits(407)
[4, 0, 7]
>>> digits(0)
[0]
```

Solution.

```
digits = lambda n: unfoldr(lambda k: NOTHING if k == 0 else (k % 10, k // 10), n)[::-1] or [0]
```

Note. The step function peels off the LEAST-significant digit first (`k % 10`) and passes what remains onward as the next seed (`k // 10`), stopping at `NOTHING` once `k` reaches 0. Because digits emerge least-significant-first, `[::-1]` is needed to restore normal reading order; ‘or `[0]`’ exists because `unfoldr(..., 0)` immediately returns an empty list (0 stops right away), yet `digits(0)` must be `[0]`, not `[]`.

3.5 Collatz flight length

challenge

Write `collatz_len(n)`: how many numbers the Collatz sequence visits from `n` down to 1, inclusive. Even halves; odd triples-plus-one.

`collatz_len(n) -> int`

```
>>> collatz_len(6)
9
>>> collatz_len(1)
1
```

Solution.

```
collatz_len = lambda n: 1 + len(unfoldr(lambda k: NOTHING if k == 1 else (k, k // 2 if k % 2 == 0 else 3 * k + 1), n))
```

Note. The step function encodes the whole rule in one line: halve `k` when it’s even, otherwise `3k + 1`, and stop (`NOTHING`) once `k` reaches 1. `unfoldr` therefore emits every number in the sequence EXCEPT the terminal 1, so `len(...)` counts those steps and `+1` accounts for the 1 that was never emitted – no mutable counter, no while loop, just an `unfold`, a length, and an addition.

Chapter 4

The List Gaps

The largest chapter because it is the least glamorous: the ordinary list and dict work that every program needs, named consistently instead of reached for ad hoc. `windows` and `groupBy` both slide a view across a sequence, but for different reasons – one for a fixed-width lookback (moving averages), one for runs of equal elements (RLE, compression). `transpose` is ragged-safe on purpose: short rows simply drop out of later columns instead of raising, so uneven data never needs a special case. `fromListWith` and `unionWith` are the two dict-building folds worth memorising – almost every “group these” or “merge these two counters” problem is one of them with the right combining function.

4.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

4.1 Run-length encoding

drill

Compress a sequence into (element, count) pairs per maximal run.

```
rle(xs) -> [(x, n)]
>>> rle('aaabbc')
[('a', 3), ('b', 2), ('c', 1)]
>>> rle([])
[]
```

Solution.

```
rle = lambda xs: [(g[0], len(g)) for g in groupBy(eq, xs)]
```

Note. `groupBy(eq, xs)` partitions `xs` into consecutive runs of equal elements (`operator.eq` supplies the comparison, so no `lambda` is needed), returning something like `[['a','a'],['b']]`. The comprehension then turns each run `g` into `(g[0], len(g))` – the repeated value and how many times it repeated – which is run-length encoding; `groupBy` has already done all the actual segmenting.

4.2 Moving average

applied

Write `mavg(n, xs)`: the `n`-point moving average of a signal.

```
mavg(n, xs) -> [float]
>>> mavg(3, [1, 2, 3, 4, 5])
[2.0, 3.0, 4.0]
>>> mavg(3, [1, 2])
[]
```

Solution.

```
mavg = lambda n, xs: [sum(w) / n for w in windows(n, xs)]
```

Note. `windows(n, xs)` produces every contiguous length-`n` slice as its own list – a genuine sliding window, not index arithmetic – and the comprehension averages each one with `sum(w) / n`. Because `windows` naturally yields zero windows when `xs` is shorter

than `n`, `mapv` returns `[]` for too-short input automatically, with no explicit length check anywhere.

4.3 Pagination

drill

Write `paginate(n, xs)`: pages of at most `n` items, last page short.

```
paginate(n, xs) -> [[x]]

>>> paginate(2, [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
```

Solution.

```
| paginate = chunksOf
```

Note. This is a pure recognition exercise: `chunksOf(n, xs)` already means “split `xs` into pages of size `n`”, with the final page possibly shorter – there is nothing left to implement. `paginate` is `chunksOf` under a different name for a different domain.

4.4 Column sums, ragged

applied

Sensor rows arrive with trailing dropouts. Sum each column over the rows that reached it – point-free.

```
colsums(rows) -> [sum]

>>> colsums([[1, 2, 3], [4, 5]])
[5, 7, 3]
>>> colsums([])
[]
```

Solution.

```
| colsums = o(list, partial(map, sum), transpose)
```

Note. `transpose(rows)` turns rows into columns, and because it’s ragged-safe, a row shorter than the others simply drops out of the columns past its own length instead of raising an error. Once the input is transposed, `map(sum, ...)` sums each resulting column, and `o` composes `transpose`, the mapping, and `list`-materialization right to left into one pipeline.

4.5 Indentation split

drill

Split a line into (leading whitespace, rest) as strings, one pass.

```
indent_split(s) -> (str, str)

>>> indent_split('  x = 1')
('  ', 'x = 1')
>>> indent_split('top')
('', 'top')
```

Solution.

```
| indent_split = lambda s: tuple(''.join(p) for p in span(str.isspace, s))
```

Note. `span(str.isspace, s)` walks `s` and splits it into (leading run of whitespace characters, everything after) as two lists of individual characters, stopping at the first non-whitespace character it meets. Because `span` works over `s` as a sequence of characters rather than as a string, “`join(...)`” is needed on each half to turn the character lists back into real strings.

Chapter 5

Maybe, Done Right

`None` is Python’s “no value”, which is exactly the problem: a function cannot then return `None` to mean a REAL answer without creating an ambiguity. `NOTHING` fixes this by being a distinct object that is never a legitimate value – `maybe_get` returns `None` for a key whose stored value truly is `None`, and `NOTHING` only when the key is missing outright. `bind` chains failable steps by short-circuiting the moment one returns `NOTHING`; `sequenceM` and `traverseM` extend that to a whole list, all-or-nothing. Once a pipeline’s failure mode is a value instead of an exception, the pipeline composes like any other function.

5.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

5.1 Safe head

drill

Write `headM(xs)`: the first element, or `NOTHING` for an empty sequence. A leading `None` is data.

`headM(xs) -> Maybe value`

```
>>> headM([7, 8])
7
>>> headM([]) is NOTHING
True
>>> headM([None]) is None
True
```

Solution.

```
headM = lambda xs: xs[0] if xs else NOTHING
```

Note. The whole exercise is what survives the check: `'xs[0] if xs else NOTHING'` returns the first element whenever the list is non-empty – even if that element genuinely IS `None`, a real stored value – and only returns the `NOTHING` sentinel when the list is truly empty. Using plain `None` to mean “empty” would make a list starting with `None` indistinguishable from an empty list, which is exactly the ambiguity the third test is built to catch.

5.2 Deep dictionary path

applied

Write `pathM(d, ks)`: follow a key path through nested dicts; `NOTHING` if any hop is missing. A stored `None` must survive the trip.

`pathM(d, ks) -> Maybe value`

```
>>> d = {'db': {'host': 'local', 'pool': None}}
>>> pathM(d, ['db', 'host'])
'local'
>>> pathM(d, ['db', 'pool']) is None
True
>>> pathM(d, ['db', 'user']) is NOTHING
True
```

Solution.

```
| pathM = lambda d, ks: foldl(lambda m, k: bind(m, lambda t: maybe_get(t, k)), ks, d)
```

Note. `foldl` threads the dictionary through each key in `ks` one at a time; at every step, `bind(m, lambda t: maybe_get(t, k))` either descends into the next nested dict (if the lookup so far succeeded) or short-circuits straight to `NOTHING` the moment any key along the path is missing. This is the `Maybe` monad’s signature move: a chain of possibly-failing steps where one failure propagates to the very end without an `if-chain` anywhere.

5.3 Salvage the parseable

drill

Given strings, keep the ones that are integers, as ints, dropping the rest.

```
salvage_ints(ss) -> [int]
```

```
>>> salvage_ints(['3', 'x', '-7', ''])
[3, -7]
```

Solution.

```
| salvage_ints = lambda ss: catMaybes([int(s) if s.lstrip('-').isdigit() and s else NOTHING for s in ss])
```

Note. Each string passes through an inline test-then-convert expression that yields either its integer value or `NOTHING` (rejecting empty strings and anything that isn’t a valid, possibly-negative integer); `catMaybes` then filters out every `NOTHING`, keeping only the strings that actually parsed. This is `mapMaybe`’s job split into two explicit, named steps.

5.4 All keys or nothing

applied

Write `need(d, ks)`: the values for `ks` in order, or `NOTHING` if ANY key is absent.

```
need(d, ks) -> Maybe [value]
```

```
>>> need({'a': 1, 'b': 2}, ['a', 'b'])
[1, 2]
>>> need({'a': 1}, ['a', 'b']) is NOTHING
True
```

Solution.

```
| need = lambda d, ks: sequenceM([maybe_get(d, k) for k in ks])
```

Note. `sequenceM` is handed a list of `Maybe` values, one `maybe_get` per requested key, and returns the fully unwrapped list only if EVERY lookup succeeded; the instant any single lookup comes back `NOTHING`, the whole result collapses to `NOTHING`. That is precisely “all keys present, or nothing at all” – no manual `all()`-style checking required.

5.5 First responder

challenge

Several config sources; write `first_set(dicts, k)`: the value from the FIRST dict that defines `k` at all – even as `None` – else `NOTHING`.

```
first_set(dicts, k) -> Maybe value
```

```
>>> first_set([{}], {'x': None}, {'x': 3}], 'x') is None
True
>>> first_set([{}], {}, 'x') is NOTHING
True
```

Solution.

```
| first_set = lambda dicts, k: mconcat(First, [maybe_get(d, k) for d in dicts])
```

Note. `mconcat` folds the list of per-dict lookups through the `First` monoid, whose combining rule is “keep the first non-`NOTHING` value seen, ignore the rest”; because `First`’s identity element IS `NOTHING`, an empty list of dicts (or one where every dict is missing the key) correctly yields `NOTHING` with no base case to write by hand.

Chapter 6

Either: Failure With a Why

Maybe answers “did it work?”; Either answers that AND “why not?”. `Ok/Err` are tagged pairs – (“ok”, `v`) or (“err”, `why`) – so `bindE` can short-circuit exactly like `bind` while still carrying the failure’s reason to wherever the pipeline is finally inspected. `sequenceE` and `traverseE` are Either’s all-or-nothing gates, and `note` is the bridge from the last chapter: it turns a `NOTHING` into an `Err` with a message attached, for the moment a `Maybe` pipeline needs to explain itself to a caller.

6.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

6.1 Lift the miss into a reason

drill

Write `get_e(d, k)`: `Ok(value)` or `Err` naming the missing key.

`get_e(d, k) -> Either`

```
>>> get_e({'a': 1}, 'a')
('ok', 1)
>>> get_e({}, 'user')
('err', "missing key: 'user'")
```

Solution.

```
get_e = lambda d, k: note(f"missing key: {k!r}", maybe_get(d, k))
```

Note. `note` is the bridge from `Maybe` to `Either`: given a message and a `Maybe` value, it turns a `NOTHING` into `Err(message)` and leaves any real value – including a successful `None` – wrapped as `Ok(value)`, unchanged. A lookup that could previously only say “missing” can now also say **WHY**.

6.2 Parse then bound

applied

Write `to_port(s)`: parse a decimal string and demand 1..65535, each failure carrying its own message.

`to_port(s) -> Either`

```
>>> to_port('8080')
('ok', 8080)
>>> to_port('http')
('err', "not a number: 'http'")
>>> to_port('99999')
('err', 'out of range: 99999')
```

Solution.

```
to_port = lambda s: bindE(
    note(f"not a number: {s!r}", int(s) if s.isdigit() else NOTHING),
    lambda n: Ok(n) if 1 <= n <= 65535 else Err(f"out of range: {n}"))
```

Note. `bindE` chains two Either-producing steps. First, `note` converts a failed `int()` parse (guarded by `isdigit()`) into an `Err` carrying a parsing message; if that succeeds, the second step (the `lambda`) checks the number falls in the valid port range and returns either `Ok(n)` or a `DIFFERENT` `Err` for an out-of-range value. Both failure reasons flow through the same `bindE` plumbing, so the caller inspects one Either at the end instead of two separate failure sites.

6.3 First error speaks

drill

Validate a batch: all `Ok` values, or the FIRST `Err` verbatim.

```
all_ports(ss) -> Either
|
| >>> all_ports(['80', '443'])
| ('ok', [80, 443])
| >>> all_ports(['80', 'x', 'y'])
| ('err', "not a number: 'x'")
```

Solution.

```
| all_ports = lambda ss: sequenceE([to_port(s) for s in ss])
```

Note. `sequenceE` applies the same all-or-nothing discipline as `sequenceM`, but for Either: it walks the list of per-string results from `to_port` and returns `Ok` of the whole list only if every one succeeded, otherwise it returns the FIRST `Err` it hit, message intact. Unlike `sequenceM`'s bare `NOTHING`, the caller learns exactly which port was invalid and why.

6.4 Division with a why

drill

Write `div_e(a, b)`: the quotient, or an error naming the offense.

```
div_e(a, b) -> Either
|
| >>> div_e(10, 4)
| ('ok', 2.5)
| >>> div_e(1, 0)
| ('err', 'division by zero')
```

Solution.

```
| div_e = lambda a, b: Err('division by zero') if b == 0 else Ok(a / b)
```

Note. This is the base case the rest of the chapter's Either machinery is built on: a genuinely failable operation that returns an `Err` with an explanation instead of raising `ZeroDivisionError`, so the failure can be caught, inspected, and composed with `bindE`/`doE` exactly like every other Either-returning step in this chapter.

Chapter 7

do-Notation

Chaining `bind` calls by hand nests one call inside the next, one level per step – the “bind pyramid”. `do` rebuilds that same chain out of an ordinary generator function: each `yield` is one `bind`, and the value sent back in is what the step would have returned. `doM` wires this to the Maybe monad, `doE` to Either – same decorator, same generator shape, different short-circuit rule. The win is not cleverness for its own sake: a five-step Maybe pipeline written with `doM` reads top to bottom, in the order the steps actually happen, with the failure path made invisible instead of handled at every line.

7.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

7.1 Config reader, flat

drill

Using `doM`, read host and port from a dict; NOTHING if either is absent.

`hostport(d) -> Maybe (host, port)`

```
>>> hostport({'host': 'h', 'port': 80})
('h', 80)
>>> hostport({'host': 'h'}) is NOTHING
True
```

Solution.

```
@doM
def hostport(d):
    h = yield maybe_get(d, 'host')
    p = yield maybe_get(d, 'port')
    return (h, p)
```

Note. `@doM` turns an ordinary generator into a Maybe pipeline: each `yield` hands a Maybe value to `doM`’s machinery, which unwraps it and sends the unwrapped value back into the generator as the result of that `yield` expression if it’s real, or aborts the WHOLE function with NOTHING the instant any `yield` produces NOTHING. Written this way, `hostport` reads as two straight-line lookups with no visible branching, even though either one could fail.

7.2 Join across two tables

applied

`users` maps id to name; `emails` maps name to address. Using `doM`, resolve an id to its address.

`address(users, emails, uid) -> Maybe str`

```
>>> u, e = {1: 'ada'}, {'ada': 'ada@x.io'}
>>> address(u, e, 1)
'ada@x.io'
>>> address(u, e, 2) is NOTHING
True
```

Solution.

```
@doM
def address(users, emails, uid):
    name = yield maybe_get(users, uid)
    return (yield maybe_get(emails, name))
```

Note. The same @doM discipline, but the second lookup’s key depends on the FIRST lookup’s result: `name = yield maybe_get(users, uid)` must succeed and bind `name` before `maybe_get(emails, name)` can even be attempted. If `uid` isn’t in `users`, the function stops at the first `yield` and never reaches the second lookup – there is nothing to explicitly check for that.

7.3 Validated record

applied

Using `doE`, build a user record from a form dict: `name` must be non-empty, `age` must parse and land in `0..130`. Each rejection explains itself.

`mk_user(form) -> Either dict`

```
>>> mk_user({'name': 'ada', 'age': '36'})
('ok', {'name': 'ada', 'age': 36})
>>> mk_user({'name': '', 'age': '36'})
('err', 'empty name')
>>> mk_user({'name': 'ada', 'age': 'old'})
('err', "bad age: 'old'")
```

Solution.

```
@doE
def mk_user(form):
    name = yield (Ok(form.get('name', '')) if form.get('name') else Err('empty name'))
    raw = form.get('age', '')
    age = yield note(f"bad age: {raw!r}", int(raw) if raw.isdigit() else NOTHING)
    yield (Ok(age) if age <= 130 else Err(f'implausible age: {age}'))
    return Ok({'name': name, 'age': age})
```

Note. This `doE` pipeline mixes `VALUE`-producing `yields` (which bind `name` and `age` for later use) with one bare, unbound `yield` – the `age <= 130` check – that runs purely for its side effect of stopping the pipeline if it fails; nothing from that check is kept, it exists only to guard what comes after. Read top to bottom, the happy path (every field valid) is the only path visible, and every rejection reason sits as an `Err` right at the line that would fail it.

7.4 Chained arithmetic

drill

Using `doE` and `div_e` from 6.4, compute $a/b + c/d$ with the first bad divisor reported.

`sum_of_ratios(a, b, c, d) -> Either`

```
>>> sum_of_ratios(1, 2, 1, 4)
('ok', 0.75)
>>> sum_of_ratios(1, 0, 1, 4)
('err', 'division by zero')
```

Solution.

```
@doE
def sum_of_ratios(a, b, c, d):
    x = yield div_e(a, b)
    y = yield div_e(c, d)
    return Ok(x + y)
```

Note. Two divisions, each already `Either`-safe via `div_e`, are bound in sequence with `doE`: `x` captures the first ratio, `y` the second, and if EITHER division is by zero, the whole function short-circuits to that division’s `Err` without ever reaching the addition. The final line is pure arithmetic with no visible error-handling code, because `doE` has already absorbed all of it.

Chapter 8

Parser Combinators

A parser is a function from a string to `(value, rest)` or `FAIL` – once that shape is fixed, parsers compose like any other function. `doP` threads the remaining input through a generator the same way `doM` threads a `Maybe`; `alt` tries alternatives in order; `many` and `sepBy` handle repetition and separator-delimited lists without recursion. `chainl1` deserves special attention: it folds a sequence of `p (op p) * LEFT`, which is exactly how left-associative arithmetic is supposed to parse, and it is the one combinator in this chapter doing real algorithmic work rather than just gluing others together.

8.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

8.1 Comma-separated integers

drill

Parse a comma-separated integer list, whitespace-blind, whole input consumed.

`int_list(s) -> Either [int]`

```
>>> int_list('1, -2 , 3')
('ok', [1, -2, 3])
>>> int_list('1, x')
('err', "unconsumed input: ' , x'")
```

Solution.

```
int_list = lambda s: runParser(sepBy(rx(r' -?\d+', int), lit(',')), s)
```

Note. `sepBy(rx(...), lit(','))` parses one or more integer tokens separated by literal commas, correctly handling the fencepost problem (`n` items need only `n - 1` separators) with no manual counting. `runParser` then additionally demands the ENTIRE input be consumed (rejecting trailing garbage) and reports the outcome as an `Either` instead of a bare value-or-`FAIL`.

8.2 key=value line

drill

Parse one 'key = value' assignment into a pair; keys are words, values run to end of line.

`kv(s) -> Either (k, v)`

```
>>> kv('host = 10.0.0.1')
('ok', ('host', '10.0.0.1'))
>>> kv(' = oops')[0]
'err'
```

Solution.

```
@doP
def _kv():
    k = yield rx(r'\w+')
    yield lit('=')
    v = yield rx(r'[\n]+', str.strip)
```

```

    return (k, v)
kvline = _kv()
kv = lambda s: runParser(kvline, s)

```

Note. Three sequential yields inside a doP block: match an identifier, match and discard a literal '=', then match everything up to end of line (stripped) as the value. @doP threads the unconsumed remainder of the string from one yield to the next automatically, so no explicit "rest of string" variable is passed by hand anywhere. Defining kvline once means the same parser value can be reused later – chapter 9's INI reader is built directly on it.

8.3 CSV row

drill

Split one unquoted CSV row into stripped fields.

csv_row(s) -> Either [str]

```

>>> csv_row('ada, 95, pass')
('ok', ['ada', '95', 'pass'])

```

Solution.

```

csv_row = lambda s: runParser(sepBy(rx(r'[^,\n]+'), str.strip), lit(','), s)

```

Note. Structurally identical to 8.1's comma-separated integers, just with a token parser that matches and strips arbitrary non-comma text instead of digits. Recognizing that a CSV row and a list of integers are the same shape with a different token is the entire exercise – sepBy itself doesn't change at all.

8.4 ISO date

applied

Parse YYYY-MM-DD into an (y, m, d) integer triple, rejecting month 13.

iso_date(s) -> Either (y, m, d)

```

>>> iso_date('2026-09-17')
('ok', (2026, 9, 17))
>>> iso_date('2026-13-01')
('err', 'bad month: 13')

```

Solution.

```

num = lambda n: rx(r'\d{%d}' % n, int)
@doP
def _date():
    y = yield num(4)
    yield lit('-')
    m = yield num(2)
    yield lit('-')
    d = yield num(2)
    return (y, m, d)
iso_date = lambda s: bindE(runParser(_date(), s),
    lambda t: Ok(t) if 1 <= t[1] <= 12 else Err(f'bad month: {t[1]}'))

```

Note. _date parses three fixed-width numeric fields (4 digits, 2 digits, 2 digits) separated by literal dashes via doP, producing a raw (year, month, day) tuple purely from the STRING SHAPE, with no awareness of what a valid month even is. bindE then takes that successfully-parsed tuple and applies a semantic check – is the month between 1 and 12 – as a separate stage, turning an otherwise-valid parse into an Err if the calendar rule is broken. Keeping "does this look like a date" and "is this a real date" as two stages means either can change without touching the other.

8.5 Sums and differences

challenge

Parse and evaluate a left-associative +/- chain over integers.

sums(s) -> Either int

```

>>> sums('10 - 3 - 2 + 1')
('ok', 6)
>>> sums('10 -')[0]
'err'

```

Solution.

```
| sums = lambda s: runParser(chain1(rx(r'[-?]\d+', int), {'+': add, '-': sub}), s)
```

Note. chain1 takes a token parser (digits) and a dict mapping operator characters to their Python functions, and folds a sequence of "number, operator, number, operator, number..." strictly left to right, matching how ordinary arithmetic associates addition and subtraction. operator.add/sub are used directly as the combining functions instead of hand-written lambdas, keeping the whole solution point-free.

Chapter 9

Monoids at Work

A monoid is nothing but an identity element paired with an associative combiner, reified as the pair `(empty, op)` – naming it as `DATA` means one engine, `mconcat`, folds every instance, and `foldMap` fuses “measure each element” with “combine the measurements” into one pass. `both` is the chapter’s real payoff: it pairs two monoids into one, so a max and a min, or a count and a total, fall out of a `SINGLE` traversal instead of two separate loops. That matters most exactly when it looks like it should not – a one-shot iterator or a huge file that can only be read once.

9.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest` referee.

9.1 Min and max, one pass

drill

Both extremes of a numeric stream in a single traversal, point-free.

```
extremes(xs) -> (min, max)

>>> extremes([3, 1, 4, 1, 5])
(1, 5)
```

Solution.

```
extremes = partial(foldMap, lambda x: (x, x), both(MinM, MaxM))
```

Note. `foldMap` first turns each element `x` into the pair `(x, x)` – one copy destined for the min side, one for the max side – and `both(MinM, MaxM)` runs `MinM`’s fold on the first half of every pair and `MaxM`’s fold on the second half, all in a `SINGLE` traversal of `xs`. That matters specifically when `xs` is a generator or a large file read once: two separate `min(xs)/max(xs)` calls would each need their own full pass, impossible for a one-shot iterator, but this needs only one.

9.2 Stream statistics

applied

Count, total and maximum of a stream in ONE pass: `((count, total), maximum)`.

```
stats(xs) -> ((n, total), mx)

>>> stats([3, 1, 4])
((3, 8), 4)
```

Solution.

```
stats = partial(foldMap, lambda x: ((1, x), x), both(both(Sum, Sum), MaxM))
```

Note. The same one-pass trick as 8.1, nested one level deeper: each element becomes `((1, x), x)`, and `both(both(Sum, Sum), MaxM)` applies `Sum` to the running count, `Sum` to the running total, and `MaxM` to the running maximum, all three statistics falling out of three nested monoids fused into a single fold. The output shape, `((n, total), mx)`, directly mirrors how the monoids were nested inside each other.

9.3 Config layering

drill

defaults, file, cli – highest layer that DEFINES a key wins, and defining it as None counts. Point-free over the layer list.

```
effective(layers, k) -> Maybe value
```

```
>>> layers = [{'v': 1}, {}, {'v': None}]
>>> effective(layers, 'v') is None
True
>>> effective(layers, 'x') is NOTHING
True
```

Solution.

```
| effective = lambda layers, k: mconcat>Last, [maybe_get(d, k) for d in layers])
```

Note. mconcat folds the list of per-layer lookups through the Last monoid, whose rule – “the most recent non-NOTHING value wins” – is precisely “later layers override earlier ones”. Because Last’s identity is NOTHING, a key present in no layer at all correctly yields NOTHING with no base case to write. This is the exact same skeleton as first_set in 5.5, with only the monoid swapped (Last instead of First) to flip which end of the list wins.

9.4 Flatten

drill

Concatenate a list of lists by monoid fold.

```
flatten(xss) -> [x]
```

```
>>> flatten([[1], [], [2, 3]])
[1, 2, 3]
```

Solution.

```
| flatten = partial(mconcat, ListM)
```

Note. mconcat(ListM, xss) folds the list of lists through ListM, the monoid whose identity is [] and whose op is list concatenation – which is precisely what “flatten a list of lists” means. partial freezes ListM in, so flatten ends up a plain one-argument function. The quadratic-cost warning matters because repeated concatenation re-copies earlier elements on every combine; fine for a handful of sublists, but itertools.chain is the right tool once there are many large ones.

Chapter 10

Capstone Labs

No new vocabulary – this chapter is the payoff for the previous eight. Each problem reaches back for tools from more than one chapter at once: parsing with `doP`, then judging the parsed value with `bindE`; folding a monoid across a batch validated with `doE`. If a chapter’s tools felt like isolated tricks on first reading, this is where they stop being isolated.

10.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

10.1 The algebraic calculator

lab

Full expression evaluator: `+ - * /`, precedence, LEFT associativity, unary minus, parentheses, decimals, whitespace; Either out.

`calc(s) -> Either float`

```
>>> calc('2 * (3 + 4) - -5')
('ok', 19.0)
>>> calc('8 - 3 - 2')
('ok', 3.0)
>>> calc('2 * (3 +')[0]
'err'
```

Solution.

```
number = rx(r'[-?\d+(\.\d+)?)', float)
@doP
def _paren():
    yield lit('(')
    v = yield expr
    yield lit(')')
    return v
unary = lambda s: alt(number, _paren())(s)
term = chainl1(unary, {'*': mul, '/': lambda a, b: a / b})
expr = chainl1(term, {'+': add, '-': sub})
calc = partial(runParser, expr)
```

Note. A small recursive-descent grammar in three layers: unary handles a bare number OR a fully parenthesized sub-expression (recurring back into `expr` for whatever is inside the parens), `term` folds a sequence of unary results left to right through `*` and `/`, and `expr` folds a sequence of `term` results left to right through `+` and `-` – together enforcing standard precedence purely through which layer calls which. `doP` threads the remaining input through every `yield` with no explicit position variable, `chainl1` does the left-associative folding, and `runParser` turns the result into an `Either` that names exactly where parsing failed. Because none of this recurses per TOKEN (only per parenthesis nesting level), a long flat expression with thousands of terms still runs in constant stack depth.

10.2 INI reader

lab

Parse [section] headers with key = value bodies into a dict of dicts.

ini(s) -> Either section: k: v

```
>>> t = '[db]\nhost = local\nport = 5432\n[app]\ndebug = true'
>>> ini(t)
('ok', {'db': {'host': 'local', 'port': '5432'}, 'app': {'debug': 'true'}})
```

Solution.

```
@doP
def _kvi():
    k = yield rx(r'\w+')
    yield lit('=')
    v = yield rx(r'[\n\[]+', str.strip)
    return (k, v)

@doP
def _section():
    yield lit('[')
    name = yield rx(r'\w+')
    yield lit(']')
    kvs = yield many(_kvi())
    return (name, dict(kvs))

ini = lambda s: bindE(runParser(many(_section()), s), o(Ok, dict))
```

Note. `_kvi` parses one key=value line, reusing the exact shape built in 8.2, and `_section` parses a bracketed header followed by ZERO OR MORE key/value lines via `many(_kvi())`; many loops rather than recurses, so an arbitrarily long – or empty – section body costs no extra stack. The top-level ini parser is `many(_section())` run through `runParser`, and `o(Ok, dict)` takes the raw list of (name, k: v) pairs on success and wraps it as name: k: v inside an `Ok`, all in one composed step.

10.3 Log triage

lab

Lines are 'LEVEL message'. In ONE pass report ((errors, total), first ERROR message or NOTHING).

triage(lines) -> ((errors, total), first_err)

```
>>> triage(['INFO up', 'ERROR db down', 'ERROR retry', 'INFO ok'])
((2, 4), 'db down')
>>> triage(['INFO up'])[1] is NOTHING
True
```

Solution.

```
tag = lambda ln: (lambda lvl, _, msg: ((lvl == 'ERROR', 1),
    msg if lvl == 'ERROR' else NOTHING))(*ln.partition(' '))
triage = partial(foldMap, tag, both(both(Sum, Sum), First))
```

Note. `tag` turns each log line into a nested tuple shaped exactly like the monoid that will consume it: `((is_it_an_error, 1), the_error_message_or_NOTHING)`. `both(both(Sum, Sum), First)` then counts errors, counts total lines, and remembers the first error's message, all as three independent monoids riding the SAME single pass over the lines. Adding a fourth statistic later means nesting one more `both` and extending `tag`'s output tuple by one field, not adding a second loop.

10.4 Batch admission

lab

Admit a batch of form dicts with 7.3's `mk_user`: all records, or the first rejection verbatim.

admit(forms) -> Either [dict]

```
>>> admit([{'name': 'ada', 'age': '36'}, {'name': 'bob', 'age': '41'}])
('ok', [{'name': 'ada', 'age': '36'}, {'name': 'bob', 'age': '41'}])
>>> admit([{'name': 'ada', 'age': '36'}, {'name': '', 'age': '2'}])
('err', 'empty name')
```

Solution.

```
admit = o(sequenceE, partial(map, mk_user), list)
```

Note. `sequenceE`, `partial(map, mk_user)`, and `list` are composed right to left with `o`: first every form in the batch runs through `mk_user`'s per-record validation from 7.3, producing a list of `Ok/Err` results; `sequenceE` then collapses that list into a single `Either` – `Ok` of every validated record if all passed, or the first `Err` encountered if any one failed. The whole batch-intake pipeline fits on one line because each piece (`mk_user`, `sequenceE`) was already built and verified independently in earlier chapters.

Part II

Compare and Contrast: Imperative vs Functional

About this part

Ten small apps, each written twice over the same inputs: once imperative, once with `haskell.py`, with a `__main__` block that ASSERTS the two agree. Every listing below is verified to actually run and agree before this book is built – what you read is what was executed.

	imperative	functional
state	mutable slots you maintain	threaded by the combinator
errors	exceptions / early return	values (NOTHING, Err) that compose
no value yet	hand-rolled sentinel per site	NOTHING, once, in the library
passes over data	one loop per concern	monoids fuse concerns into one pass
extension	edit inside the loop	add a stage / another <code>both</code>

Chapter 11

calculator

evaluate '2 * (3 + 4) - 5' both ways

Imperative

```
def calc_imp(s):
    toks, i = [], 0
    while i < len(s):
        c = s[i]
        if c.isspace(): i += 1; continue
        if c in '+-*/()': toks.append(c); i += 1; continue
        if c.isdigit():
            j = i
            while j < len(s) and (s[j].isdigit() or s[j] == '.'): j += 1
            toks.append(float(s[i:j])); i = j; continue
        return ('err', f'bad char {c!r}')

    class P:
        pos = 0
    def peek(): return toks[P.pos] if P.pos < len(toks) else None
    def unary():
        t = peek()
        if t == '-': P.pos += 1; return -unary()
        if t == '(':
            P.pos += 1; v = expr()
            if peek() != ')': raise ValueError('expected ')
            P.pos += 1; return v
        if isinstance(t, float): P.pos += 1; return t
        raise ValueError(f'unexpected {t!r}')
    def term():
        v = unary()
        while peek() in ('*', '/'):
            op = peek(); P.pos += 1; w = unary()
            v = v * w if op == '*' else v / w
        return v
    def expr():
        v = term()
        while peek() in ('+', '-'):
            op = peek(); P.pos += 1; w = term()
            v = v + w if op == '+' else v - w
        return v

    try:
        v = expr()
        return ('ok', v) if P.pos == len(toks) else ('err', 'unconsumed input')
    except ValueError as e:
        return ('err', str(e))
```

Functional

```

number = rx(r'~?\d+(\.\d+)?', float)
@doP
def _paren():
    yield lit('(')
    v = yield expr_fn
    yield lit(')')
    return v
unary_fn = lambda s: alt(number, _paren())(s)
term_fn = chainl1(unary_fn, {'*': mul, '/': lambda a, b: a / b})
expr_fn = chainl1(term_fn, {'+': add, '-': sub})
calc_fn = partial(runParser, expr_fn)

```

Discussion

The imperative version's mutable cursor (`class P: pos = 0`) is a common workaround for parsing in Python: state lives on an object because a closure can't reassign an outer `int` without `nonlocal` threaded through every helper. The functional version needs no cursor at all – `doP` threads the remaining input as an ordinary return value, and `chainl1` handles left-associativity as a fold instead of a hand-written `while` loop. Errors ride two different rails, too: exceptions in the imperative version, an `Either` in the functional one – and only the `Either` can be inspected, logged, or handed to another function without a `try/except` at every call site.

Example

The demo below is the file's own `__main__` block; the output beneath it is that exact run's actual captured `stdout` – not retyped, not fabricated.

```

if __name__ == "__main__":
    cases = ['2 * (3 + 4) - -5', '8 - 3 - 2', '10 / 4', '1 + (2', '1 )']
    for s in cases:
        a, b = calc_imp(s), calc_fn(s)
        assert a[0] == b[0] and (a[0] == 'err' or a[1] == b[1]), (s, a, b)
        print(f"{s!r:22s} imp={a} fn={b}")
    print("calculator: both agree on all cases")

```

Output.

```

'2 * (3 + 4) - -5'      imp=('ok', 19.0)  fn=('ok', 19.0)
'8 - 3 - 2'            imp=('ok', 3.0)    fn=('ok', 3.0)
'10 / 4'               imp=('ok', 2.5)    fn=('ok', 2.5)
'1 + (2'               imp=('err', 'expected ') fn=('err', "unconsumed input: '+' (2'")
'1 )'                 imp=('err', 'unconsumed input') fn=('err', "unconsumed input: ')'")
calculator: both agree on all cases

```

Chapter 12

gradebook

validate 'name,score' rows, then one-pass class stats

Imperative

```
def admit_imp(rows):
    out = []                                # accumulate + early return
    for row in rows:
        name, _, raw = row.partition(',')
        if not name:
            return ('err', f'empty name in {row!r}')
        if not raw.strip().isdigit():
            return ('err', f'bad score in {row!r}')
        score = int(raw)
        if score > 100:
            return ('err', f'score > 100 in {row!r}')
        out.append((name, score))
    n = tot = 0                             # second loop for stats
    best = float('-inf')
    for _, s in out:
        n += 1; tot += s
        if s > best: best = s
    return ('ok', (out, (n, tot), best))
```

Object-Oriented

```
class GradeBook:
    """The same job, encapsulated: a mutable object that grows by .add(row),
    raising on the first bad row instead of returning an Err tuple."""
    def __init__(self):
        self.rows = []

    def add(self, row):
        name, _, raw = row.partition(',')
        if not name:
            raise ValueError(f'empty name in {row!r}')
        if not raw.strip().isdigit():
            raise ValueError(f'bad score in {row!r}')
        score = int(raw)
        if score > 100:
            raise ValueError(f'score > 100 in {row!r}')
        self.rows.append((name, score))
        return self                        # chainable: book.add(r1).add(r2)

    def stats(self):
```

```

        n = len(self.rows)
        tot = sum(s for _, s in self.rows)
        best = max((s for _, s in self.rows), default=float('-inf'))
        return (n, tot), best

def admit_oop(rows):
    book = GradeBook()
    try:
        for row in rows:
            book.add(row)
    except ValueError as e:
        return ('err', str(e))
    return ('ok', (book.rows, *book.stats()))

```

Functional

```

@doE
def _row(row):
    name, _, raw = row.partition(',')
    yield Ok(name) if name else Err(f'empty name in {row!r}')
    s = yield note(f'bad score in {row!r}', int(raw) if raw.strip().isdigit() else NOTHING)
    yield Ok(s) if s <= 100 else Err(f'score > 100 in {row!r}')
    return Ok((name, s))

stats      = partial(foldMap, lambda p: ((1, p[1]), p[1]), both(both(Sum, Sum), MaxM))
admit_fn = lambda rows: bindE(traverseE(_row, rows),
                              lambda ps: Ok((ps, *stats(ps))))

```

Discussion

Compare all three. The imperative version accumulates into a list and returns early on a bad row, then needs a SECOND loop afterward just to compute stats. The object-oriented version is the natural instinct for validation-heavy code – a class that grows by `.add()` and raises on trouble – but it still needs a `try/except` at the call site, and its `.stats()` re-walks the same rows the loop already built. The functional version fuses validation (`traverseE`) and the one-pass fold (`foldMap` with `both` nested twice) into two lines: one pass builds the valid rows AND the stats at once, because `both` fuses monoids instead of running loops back to back.

Example

The demo below is the file's own `__main__` block; the output beneath it is that exact run's actual captured `stdout` – not retyped, not fabricated.

```

if __name__ == "__main__":
    for rows in (ROWS, BAD):
        a, b, c = admit_imp(rows), admit_fn(rows), admit_oop(rows)
        assert a == b == c, (rows, a, b, c)
        print(("ok  :" if a[0] == "ok" else "err :"), b)
    print("gradebook: all three agree")

```

Output.

```

ok  : ('ok', ((('ada', 95), ('bob', 71), ('eve', 88)), (3, 254), 95))
err : ('err', "empty name in ',71'")
gradebook: all three agree

```

Chapter 13

layers

layered config: defaults < file < cli; None is a real value

Imperative

```
_UNSET = object()                                # invent a private sentinel,
def effective_imp(layers, k):                      # because None must survive
    val = _UNSET
    for d in layers:
        if k in d:
            val = d[k]
    return NOTHING if val is _UNSET else val

def merged_imp(layers):
    out = {}
    for d in layers:
        for k, v in d.items():
            out[k] = v
    return out
```

Functional

```
effective_fn = lambda layers, k: mconcat(Last, [maybe_get(d, k) for d in layers])
merged_fn    = lambda layers: foldl(lambda a, b: unionWith(lambda old, new: new, a, b),
                                   layers, {})
```

Discussion

This is the one example where “just use a config object” would recreate the exact bug the Maybe chapter exists to prevent: a plain class doing `self.proxy = self.proxy` or `default` silently treats an explicit `None` as “no override” – backwards from what this problem needs, since `FILECFG`’s `proxy: None` must WIN over the default, not defer to it. The imperative fix (a private `_UNSET` sentinel) and the functional fix (`NOTHING`) are the same idea; object-oriented encapsulation doesn’t make the ambiguity go away, it just relocates where you’d have to solve it.

Example

The demo below is the file’s own `__main__` block; the output beneath it is that exact run’s actual captured `stdout` – not retyped, not fabricated.

```
if __name__ == "__main__":
    for k in ['host', 'port', 'proxy', 'nope']:
```

```
a, b = effective_imp(LAYERS, k), effective_fn(LAYERS, k)
assert a is b or a == b, (k, a, b)
print(f"{k:6s} -> {b!r}")
assert merged_imp(LAYERS) == merged_fn(LAYERS)
assert effective_fn(LAYERS, 'proxy') is None      # the disable survives
print("layers: both agree; explicit None beat the default")
```

Output.

```
host    -> '10.0.0.1'
port    -> 9000
proxy   -> None
nope    -> NOTHING
layers: both agree; explicit None beat the default
```

Chapter 14

leaderboard

clamp scores into [0, 100], rank by score desc, name asc

Imperative

```
def rank_imp(rows):
    clamped = []
    for name, score in rows:
        if score < 0:
            score = 0
        elif score > 100:
            score = 100
        clamped.append((name, score))
    clamped.sort(key=lambda r: (-r[1], r[0]))      # score desc, name asc
    return clamped
```

Object-Oriented

```
class Player:
    def __init__(self, name, score):
        self.name = name
        self.score = max(0, min(100, score))      # clamp at construction time

class Leaderboard:
    def __init__(self):
        self.players = []

    def add(self, name, score):
        self.players.append(Player(name, score))
        return self

    def ranked(self):
        ordered = sorted(self.players, key=lambda p: (-p.score, p.name))
        return [(p.name, p.score) for p in ordered]

def rank_oop(rows):
    board = Leaderboard()
    for name, score in rows:
        board.add(name, score)
    return board.ranked()
```

Functional

```

clamp = lambda lo, hi: o(partial(max, lo), partial(min, hi))
clamp100 = clamp(0, 100)
name_of, score_of = itemgetter(0), itemgetter(1)
rank_fn = lambda rows: sortOn(
    lambda r: (-score_of(r), name_of(r)),
    [(name_of(r), clamp100(score_of(r))) for r in rows])

```

Discussion

All three agree, so look at WHERE each one clamps. The imperative version clamps inside the loop with an `if/elif`. The object-oriented version clamps once, in `Player.__init__` – so a `Player` object is a standing proof that its own score is always valid, which is real encapsulation value, not just different syntax for the same check. The functional version clamps by composing two curried `max/min` calls into one reusable function (`clamp`) – no class needed to get the same “validate once, trust everywhere” property, because `clamp100` IS the validated boundary, not an object that happens to carry one.

Example

The demo below is the file’s own `__main__` block; the output beneath it is that exact run’s actual captured `stdout` – not retyped, not fabricated.

```

if __name__ == "__main__":
    a, b, c = rank_imp(ROWS), rank_fn(ROWS), rank_oop(ROWS)
    assert a == b == c, (a, b, c)
    print("ranked:", b)
    print("leaderboard: all three agree")

```

Output.

```

ranked: [('ada', 100), ('cy', 82), ('dee', 82), ('ed', 47), ('bob', 0)]
leaderboard: all three agree

```

Chapter 15

logtrriage

'LEVEL message' lines: error count, total, first error message

Imperative

```
def triage_imp(lines):
    errors = 0                                # three mutable slots
    total = 0
    first = None                              # and a hand-rolled sentinel:
    seen_first = False                        # None alone can't mean "none yet"
    for ln in lines:                          # if a message could be empty
        lvl, _, msg = ln.partition(' ')
        total += 1
        if lvl == 'ERROR':
            errors += 1
            if not seen_first:
                first, seen_first = msg, True
    return ((errors, total), first if seen_first else NOTHING)
```

Functional

```
tag = lambda ln: (lambda lvl, _, msg:
    ((lvl == 'ERROR', 1), msg if lvl == 'ERROR' else NOTHING)
   )(*ln.partition(' '))
triage_fn = partial(foldMap, tag, both(both(Sum, Sum), First))
```

Discussion

Two mutable slots plus a hand-rolled `seen_first` flag in the imperative version, because plain `None` can't distinguish “no error yet” from “the first error had an empty message”. The functional version needs no flag at all: `First`'s identity is `NOTHING`, not `None`, so the fold itself carries the distinction. This is the same bug shape as `layers.py` – `first/seen_first` is a private sentinel invented from scratch, precisely because Python's `None` is overloaded.

Example

The demo below is the file's own `__main__` block; the output beneath it is that exact run's actual captured `stdout` – not retyped, not fabricated.

```
if __name__ == "__main__":
    logs = ['INFO up', 'ERROR db down', 'WARN slow', 'ERROR retry', 'INFO ok']
    a, b = triage_imp(logs), triage_fn(logs)
```

```
assert a == b, (a, b)
print("triage:", b)
assert triage_imp([])[1] is NOTHING and triage_fn([])[1] is NOTHING
print("logtriage: both agree, empty log included")
```

Output.

```
triage: ((2, 5), 'db down')
logtriage: both agree, empty log included
```

Chapter 16

orderbatch

capstone: validate a batch of orders ($qty > 0$, $price > 0$), computing each line total; the whole batch fails at the first bad order, carrying which one and why

Imperative

```
def total_imp(order):
    if order["qty"] <= 0:
        return ("err", f"{order['item']}: bad qty {order['qty']}")
    if order["price"] <= 0:
        return ("err", f"{order['item']}: bad price {order['price']}")
    return ("ok", round(order["qty"] * order["price"], 2))

def admit_imp(orders):
    out = []
    for o in orders:
        r = total_imp(o)
        if r[0] == "err":
            return r
        out.append(r[1])
    return ("ok", out)
```

Functional

```
@doE
def total_fn(order):
    qty = yield (Ok(order["qty"]) if order["qty"] > 0
                 else Err(f"{order['item']}: bad qty {order['qty']}"))
    price = yield (Ok(order["price"]) if order["price"] > 0
                   else Err(f"{order['item']}: bad price {order['price']}"))
    return Ok(round(qty * price, 2))

admit_fn = lambda orders: sequenceE([total_fn(o) for o in orders])
```

Discussion

Deliberately no object-oriented version here: an `Order` class with a `.total()` method would just move `total_imp`'s body into a method, changing nothing about how a validation failure PROPAGATES through a whole batch. That propagation – stop at the first bad order, carry which one and why – is what `doE` and `sequenceE` actually buy; encapsulating the per-order check doesn't touch the batch-level control flow, which is the actual hard part of this problem.

Example

The demo below is the file's own `__main__` block; the output beneath it is that exact run's actual captured `stdout` – not retyped, not fabricated.

```
if __name__ == "__main__":
    for orders in (ORDERS_OK, ORDERS_BAD):
        a, b = admit_imp(orders), admit_fn(orders)
        assert a == b, (a, b)
    print("good batch:", admit_fn(ORDERS_OK))
    print("bad batch: ", admit_fn(ORDERS_BAD))
    print("orderbatch: both agree")
```

Output.

```
good batch: ('ok', [4.5, 8.0])
bad batch:  ('err', 'cup: bad qty 0')
orderbatch: both agree
```

Chapter 17

orgchart

resolve an employee’s manager’s manager’s email, or NOTHING if any hop is missing. None-as-Nothing means a missing hop and a real missing-email must stay distinguishable

Imperative

```
def skip_manager_email_imp(emp):
    mgr = MANAGER_OF.get(emp, NOTHING)
    if mgr is NOTHING:
        return NOTHING
    skip = MANAGER_OF.get(mgr, NOTHING)
    if skip is NOTHING:
        return NOTHING
    if skip not in EMAIL_OF:
        return NOTHING
    return EMAIL_OF[skip]
```

Functional

```
@doM
def skip_manager_email_fn(emp):
    mgr = yield maybe_get(MANAGER_OF, emp)
    skip = yield maybe_get(MANAGER_OF, mgr)
    return (yield maybe_get(EMAIL_OF, skip))
```

Discussion

Also deliberately no object-oriented version: wrapping MANAGER_OF/EMAIL_OF in a class would just rename maybe_get/doM as methods, and the real challenge – keeping “employee not found” distinguishable from “employee found, no email on file” – lives in the DATA MODEL, not the calling convention. A hypothetical .skip_manager_email() written with plain dict.get(x) returning None would silently reintroduce the exact ambiguity NOTHING exists to prevent; you would just end up reimplementing a sentinel inside the class anyway.

Example

The demo below is the file’s own __main__ block; the output beneath it is that exact run’s actual captured stdout – not retyped, not fabricated.

```
if __name__ == "__main__":
    for emp in ("ada", "grace", "nobody"):
        a, b = skip_manager_email_imp(emp), skip_manager_email_fn(emp)
```

```
    assert a == b, (emp, a, b)
print("ada's skip-manager email:", skip_manager_email_fn("ada"))
print("grace's skip-manager email is NOTHING:",
      skip_manager_email_fn("grace") is NOTHING)
print("orgchart: both agree")
```

Output.

```
ada's skip-manager email: None
grace's skip-manager email is NOTHING: True
orgchart: both agree
```

Chapter 18

payoff

loan payoff schedule: balances after each fixed payment, and how many payments until paid off

Imperative

```
def schedule_imp(balance, payment, rate):
    out, n = [balance], 0
    while balance > 0:
        balance = round(balance * (1 + rate) - payment, 2)
        if balance < 0:
            balance = 0.0
        out.append(balance)
        n += 1
    return out, n
```

Functional

```
def _step(payment, rate):
    def step(balance):
        if balance <= 0:
            return NOTHING
        nxt = round(balance * (1 + rate) - payment, 2)
        nxt = max(nxt, 0.0)
        return (nxt, nxt)
    return step

def schedule_fn(balance, payment, rate):
    rest = unfoldr(_step(payment, rate), balance)
    return [balance] + rest, len(rest)
```

Discussion

The imperative `while` loop and the functional `unfoldr` compute the same recurrence, but `unfoldr`'s stopping condition is DATA (NOTHING) returned from the step function, not a loop condition checked from outside. That distinction matters most for testing: `step(payment, rate)` is a pure function you can call directly with one balance and inspect the answer, with no loop, no mutable balance variable, and no need to run the whole schedule just to check one step's arithmetic.

Example

The demo below is the file's own `__main__` block; the output beneath it is that exact run's actual captured `stdout` – not retyped, not fabricated.

```
if __name__ == "__main__":  
    a, b = schedule_imp(1000, 120, 0.02), schedule_fn(1000, 120, 0.02)  
    assert a == b, (a, b)  
    hist, n = b  
    print(f"paid off in {n} payments:", hist)  
    print("payoff: both agree")
```

Output.

```
paid off in 10 payments: [1000, 900.0, 798.0, 693.96, 587.84, 479.6, 369.19, 256.57, 141.7, 24.53, 0.0]  
payoff: both agree
```

Chapter 19

sensoralert

smooth a sensor stream with a 3-wide moving average, then report each consecutive run of over-threshold readings as (start, len)

Imperative

```
def alerts_imp(xs, n, threshold):
    smoothed = []
    for i in range(len(xs) - n + 1):
        window = xs[i:i + n]
        smoothed.append(sum(window) / n)
    flags = [v > threshold for v in smoothed]
    runs, i = [], 0
    while i < len(flags):
        if flags[i]:
            start = i
            while i < len(flags) and flags[i]:
                i += 1
            runs.append((start, i - start))
        else:
            i += 1
    return runs
```

Functional

```
def alerts_fn(xs, n, threshold):
    smoothed = [sum(w) / n for w in windows(n, xs)]
    flagged = [(i, v > threshold) for i, v in enum(smoothed)]
    groups = groupBy(lambda a, b: a[1] == b[1], flagged)
    return [(g[0][0], len(g)) for g in groups if g[0][1]]
```

Discussion

`windows` and `groupBy` each replace a hand-rolled index-walking loop with a name. The imperative version’s inner `while` (walking `flags` to find where each run ends) is exactly what `groupBy` does generically – once “runs of the same flag value” is recognised as a `groupBy` problem, the run-finding code disappears entirely, replaced by grouping and then filtering for the runs that are `True`.

Example

The demo below is the file’s own `__main__` block; the output beneath it is that exact run’s actual captured `stdout` – not retyped, not fabricated.

```
if __name__ == "__main__":  
    a, b = alerts_imp(READINGS, 3, THRESHOLD), alerts_fn(READINGS, 3, THRESHOLD)  
    assert a == b, (a, b)  
    print("alert runs (start, length):", b)  
    print("sensoralert: both agree")
```

Output.

```
alert runs (start, length): [(1, 3), (5, 3)]  
sensoralert: both agree
```

Chapter 20

wordfreq

top-N word frequencies, ties alphabetical

Imperative

```
def top_imp(n, text):
    counts = {}
    for w in text.lower().split():
        if w in counts:
            counts[w] += 1
        else:
            counts[w] = 1
    items = list(counts.items())
    items.sort(key=lambda kv: (-kv[1], kv[0]))
    return items[:n]
```

mutate a dict

then sort, then slice

Functional

```
top_fn = lambda n, text: pipe(
    text, str.lower, words,
    lambda ws: fromListWith(add, [(w, 1) for w in ws]),
    dict.items, partial(sortOn, lambda kv: (-kv[1], kv[0])),
    partial(take, n))
```

Discussion

`pipe` reads top to bottom in the order the data actually flows – lowercase, split, count, sort, take – while the imperative version’s five separate steps (build counts, get items, sort in place, slice) are the same five stages with no name tying them together. Neither version is shorter by much; the functional one is easier to extend, because inserting a new stage means inserting one more line in the `pipe` call, not renumbering a sequence of loop variables.

Example

The demo below is the file’s own `__main__` block; the output beneath it is that exact run’s actual captured `stdout` – not retyped, not fabricated.

```
if __name__ == "__main__":
    a, b = top_imp(3, TEXT), top_fn(3, TEXT)
    assert a == b, (a, b)
    print("top 3:", b)
    print("wordfreq: both agree")
```

Output.

```
top 3: [('the', 3), ('fox', 2), ('brown', 1)]
wordfreq: both agree
```

Part III

Appendices

Appendix A

haskell.py, Complete

```
1  """haskell.py -- the comprehensive Haskell-flavoured toolbox for Python.
2
3  One importable vocabulary: the working parts of the Prelude, Data.List, Data.Maybe, Data.Either,
4 Data.Map, Data.Function, Control.Monad, monoids, and parser combinators, folded into sections and
5 mastered as one language.
6
7  Design rules
8      * Where the stdlib already has it, the Haskell name WRAPS the stdlib (scanl is
9        itertools.accumulate, memo is functools.cache) -- one vocabulary, zero reimplementations.
10     * Failure is a value. Maybe uses the true NOTHING sentinel, so None is LEGAL data everywhere;
11       parsers fail with FAIL, and Either carries why.
12     * do-notation via generators: `yield` is >=> (linear monads only).
13     * Constant stack wherever Haskell would lean on TCO: loops, not recursion.
14     * Python keywords and builtins keep prelude.py's underscore convention (not_, map_, filter_,
15       zip_); `id` stays Python's -- use identity.
16 """
17 from functools import reduce as _reduce, partial, cache as memo, cmp_to_key
18 from itertools import accumulate as _acc, islice as _islice, pairwise as _pairwise
19 from itertools import chain, count, cycle, repeat
20 from operator import add, sub, mul, not_, eq
21 from math import gcd, lcm, hypot, isqrt, prod as product
22 from collections import Counter, defaultdict, deque
23 from heapq import heappush, heappop, merge as _hmerge
24 import re as _re
25
26 # ===== 1. functions =====
27 identity = lambda x: x
28 const    = lambda x: lambda _: x
29 flip     = lambda f: lambda a, b: f(b, a)
30 curry    = lambda f: lambda x: lambda y: f(x, y)
31 uncurry  = lambda f: lambda p: f(*p)
32
33 def compose(*fs):
34     """Right-to-left composition, iterative (constant stack).
35     >>> compose(str, abs)(-3)
36     '3'
37     >>> o(str, abs)(-3)
38     '3'
39     """
40     def g(x):
41         for f in reversed(fs):
42             x = f(x)
43         return x
44     return g
45
46 o = compose                                # ML's operator: o(f, g) is f after g
47
48 def pipe(x, *fs):
49     """Left-to-right application: pipe(x, f, g) == g(f(x)).
50     >>> pipe(-3, abs, str)
51     '3'
52     """
53     for f in fs:
54         x = f(x)
```

```

55     return x
56
57 def on(f, g):
58     """Data.Function: combine two arguments via a key.
59     >>> on(sub, len)('haskell', 'py')
60     5
61     """
62     return lambda x, y: f(g(x), g(y))
63
64 # ===== 2. pairs =====
65 fst = lambda p: p[0]
66 snd = lambda p: p[1]
67 swap = o(tuple, reversed)
68
69 # ===== 3. sentinels =====
70 class _Sentinel:
71     __slots__ = ("_n",)
72     def __init__(self, n): self._n = n
73     def __repr__(self): return self._n
74     def __bool__(self): return False
75
76 NOTHING = _Sentinel("NOTHING") # Maybe's Nothing; None is a legal Just here
77 FAIL = _Sentinel("FAIL") # parser failure; (None, rest) is a legal success
78 _MISS = _Sentinel("_MISS") # internal: absent optional argument
79
80 # ===== 4. arithmetic & logic =====
81 # add, sub, mul, not_ are operator's; gcd, lcm, hypot, isqrt, product are math's
82 div = lambda a, b: a // b # floors, like Haskell div
83 mod = lambda a, b: a % b # sign follows b
84 halve = lambda n: n // 2
85 even = lambda n: n % 2 == 0
86 odd = lambda n: n % 2 == 1
87 otherwise = True
88
89 def quot(a, b):
90     """Truncating division, Haskell quot; rem's sign follows a.
91     >>> quotRem(-7, 2)
92     (-3, -1)
93     """
94     return -(-a // b) if (a < 0) != (b < 0) else a // b
95 rem = lambda a, b: a - b * quot(a, b)
96 quotRem = lambda a, b: (quot(a, b), rem(a, b))
97 signum = lambda x: (x > 0) - (x < 0)
98 succ = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1
99 pred = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1
100
101 # ===== 5. folds, scans, unfolds =====
102 def foldl(f, xs, base=_MISS):
103     """foldl; seeds with the first element if no base is given.
104     >>> foldl(lambda a, b: a - b, [1, 2, 3], 10)
105     4
106     """
107     return _reduce(f, xs) if base is _MISS else _reduce(f, xs, base)
108
109 def foldr(f, xs, base):
110     """Right fold without recursion.
111     >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
112     [1, 2, 3]
113     """
114     return _reduce(flip(f), reversed(list(xs)), base)
115
116 foldl1 = foldl
117 foldr1 = lambda f, xs: _reduce(flip(f), reversed(list(xs)))
118
119 def scanl(f, xs, base=_MISS):
120     """All intermediate foldl states (itertools.accumulate).
121     >>> scanl(add, [1, 2, 3], 0)
122     [0, 1, 3, 6]
123     """
124     return list(_acc(xs, f) if base is _MISS else _acc(xs, f, initial=base))
125
126 scanl1 = lambda f, xs: scanl(f, xs)
127 scanr = lambda f, xs, base: scanl(flip(f), reversed(list(xs)), base)[::-1]
128 scanr1 = lambda f, xs: scanl(flip(f), reversed(list(xs)))[::-1]

```

```

129
130 def unfoldr(f, seed):
131     """Anamorphism: f(seed) -> (value, seed') or NOTHING to stop.
132     >>> unfoldr(lambda n: NOTHING if n == 0 else (n, n - 1), 4)
133     [4, 3, 2, 1]
134     """
135     out = []
136     while (r := f(seed)) is not NOTHING:
137         out.append(r[0]); seed = r[1]
138     return out
139
140 def until(cond, f, x):
141     """Loop f until cond holds.
142     >>> until(lambda n: n > 100, lambda n: n * 2, 1)
143     128
144     """
145     while not cond(x):
146         x = f(x)
147     return x
148
149 # brute-force enumerations (say the bound out loud):
150 replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]]) # |xs|^n
151 subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]]) # 2^n
152
153 # ===== 6. streams ===== (lazy; chain, count, cycle, repeat are itertools')
154 def iterate(f, x):
155     """Infinite stream x, f(x), f(f(x)), ...
156     >>> take(5, iterate(lambda n: n * 2, 1))
157     [1, 2, 4, 8, 16]
158     """
159     while True:
160         yield x
161         x = f(x)
162
163 take = lambda n, xs: list(_islice(iter(xs), n)) # safe on infinite streams
164
165 # ===== 7. list basics =====
166 head = lambda xs: xs[0]
167 tail = lambda xs: xs[1:]
168 init = lambda xs: xs[:-1]
169 last = lambda xs: xs[-1]
170 drop = lambda n, xs: list(xs)[max(n, 0):]
171 splitAt = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs)
172 replicate = lambda n, x: [x] * n
173 elem = lambda x, xs: x in xs
174 notElem = lambda x, xs: x not in xs
175 null = lambda xs: len(xs) == 0
176 nub = lambda xs: o(list, dict.fromkeys(xs)) # unique, first seen wins
177 enum = lambda xs, start=0: list(enumerate(xs, start))
178 pairwise = lambda xs: list(_pairwise(xs))
179 zip_ = lambda a, b: list(zip(a, b))
180 zip3 = lambda a, b, c: list(zip(a, b, c))
181 unzip = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], [])
182 isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p)
183 isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p)
184
185 def lookup(k, pairs):
186     """Assoc-list lookup into Maybe (a stored None is data).
187     >>> lookup('b', [('a', 1), ('b', None)]) is None
188     True
189     >>> lookup('c', [('a', 1)]) is NOTHING
190     True
191     """
192     return next((v for kk, v in pairs if kk == k), NOTHING)
193
194 def stripPrefix(p, xs):
195     """The rest after prefix p, or NOTHING.
196     >>> stripPrefix('foo', 'foobar')
197     'bar'
198     >>> stripPrefix('x', 'foobar') is NOTHING
199     True
200     """
201     return xs[len(p):] if isPrefixOf(p, xs) else NOTHING
202

```

```

203 # ===== 8. higher-order lists =====
204 map_      = lambda f, xs: [f(x) for x in xs]
205 filter_   = lambda cond, xs: [x for x in xs if cond(x)]
206 concat    = lambda xss: [x for xs in xss for x in xs]
207 concatMap = lambda f, xs: [y for x in xs for y in f(x)]
208 cross     = lambda a, b: [(x, y) for x in a for y in b]
209 starmap   = lambda f, pairs: [f(*p) for p in pairs]
210 zipWith   = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]
211 zipWith3  = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)]
212
213 def find(cond, xs):
214     """First match, lazily, into Maybe.
215     >>> find(even, [1, 3, 4, 5])
216     4
217     >>> find(even, [1, 3]) is NOTHING
218     True
219     """
220     return next((x for x in xs if cond(x)), NOTHING)
221
222 def partition(cond, xs):
223     """(keepers, rest) in ONE pass, cond called once per element.
224     >>> partition(even, [1, 2, 3, 4])
225     ([2, 4], [1, 3])
226     """
227     yes, no = [], []
228     for x in xs:
229         (yes if cond(x) else no).append(x)
230     return (yes, no)
231
232 # ===== 9. slicing & spans =====
233 def span(cond, xs):
234     """Split at first failure, one pass, one cond call per element.
235     >>> span(lambda x: x < 3, [1, 2, 3, 1])
236     ([1, 2], [3, 1])
237     """
238     xs = list(xs)
239     i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
240     return (xs[:i], xs[i:])
241
242 break_ = lambda cond, xs: span(o(not_, cond), xs)
243
244 def takewhile(cond, xs):
245     # generator
246     for x in xs:
247         if not cond(x):
248             return
249         yield x
250
251 def dropwhile(cond, xs):
252     # generator
253     it = iter(xs)
254     for x in it:
255         if not cond(x):
256             yield x
257             break
258     yield from it
259
260 def takeuntil(cond, xs):
261     # takewhile that INCLUDES the first hit
262     for x in xs:
263         yield x
264         if cond(x):
265             return
266
267 takeWhile = lambda cond, xs: list(takewhile(cond, xs))
268 dropWhile = lambda cond, xs: list(dropwhile(cond, xs))
269
270 inits     = lambda xs: [xs[:i] for i in range(len(xs) + 1)]
271 tails    = lambda xs: [xs[i:] for i in range(len(xs) + 1)]
272
273 def chunksOf(n, xs):
274     """
275     >>> chunksOf(2, [1, 2, 3, 4, 5])
276     [[1, 2], [3, 4], [5]]
277     """
278     xs = list(xs)
279     return [xs[i:i + n] for i in range(0, len(xs), n)]
280

```

```

277 def windows(n, xs):
278     """Sliding windows of width n.
279     >>> windows(3, [1, 2, 3, 4])
280     [[1, 2, 3], [2, 3, 4]]
281     """
282     xs = list(xs)
283     return [xs[i:i + n] for i in range(len(xs) - n + 1)]
284
285 def groupBy(eq, xs):
286     """Haskell semantics: runs anchored on each run's FIRST element.
287     >>> groupBy(lambda a, b: a == b, "aabbba")
288     [['a', 'a'], ['b', 'b'], ['a']]
289     """
290     out = []
291     for x in xs:
292         if out and eq(out[-1][0], x):
293             out[-1].append(x)
294         else:
295             out.append([x])
296     return out
297
298 group = partial(groupBy, eq)
299
300 def transpose(rows):
301     """Ragged-safe like Data.List.
302     >>> transpose([[1, 2, 3], [4, 5]])
303     [[1, 4], [2, 5], [3]]
304     """
305     rows = [list(r) for r in rows]
306     out, i = [], 0
307     while any(i < len(r) for r in rows):
308         out.append([r[i] for r in rows if i < len(r)])
309         i += 1
310     return out
311
312 # ===== 10. sorting & searching =====
313 sortOn = lambda f, xs: sorted(xs, key=f)
314 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp))
315 maxOn = lambda f, xs: max(xs, key=f)
316 minOn = lambda f, xs: min(xs, key=f)
317 merge = lambda a, b: list(_hmerge(a, b))          # stable merge of sorted lists
318
319 # ===== 11. strings =====
320 lines = str.splitlines
321 unlines = lambda ls: "".join(l + "\n" for l in ls)
322 words = str.split
323 unwords = " ".join
324
325 # ===== 12. containers ===== (Counter, defaultdict, deque, heappush/heappop are stdlib's)
326 def fromListWith(f, pairs):
327     """Data.Map: THE dict-building fold; f(new, old).
328     >>> fromListWith(add, [('a', 1), ('b', 2), ('a', 3)]) == {'a': 4, 'b': 2}
329     True
330     """
331     d = {}
332     for k, v in pairs:
333         d[k] = f(v, d[k]) if k in d else v
334     return d
335
336 def insertWith(f, k, v, d):
337     """Persistent insert (fresh dict); f(new, old).
338     >>> insertWith(add, 'a', 5, {'a': 1}) == {'a': 6}
339     True
340     """
341     return {**d, k: f(v, d[k]) if k in d else v}
342
343 def unionWith(f, a, b):
344     """Data.Map: merge two dicts, combining shared keys with f.
345     >>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4}) == {'a': 1, 'b': 5, 'c': 4}
346     True
347     """
348     return {k: f(a[k], b[k]) if k in a and k in b else (a[k] if k in a else b[k])
349             for k in a.keys() | b.keys()}
350

```

```

351 def getpath(t, ks, default=NOTHING):
352     """Read along a key path without autovivifying; Maybe out.
353     >>> getpath({'a': {'b': None}}, ['a', 'b']) is None
354     True
355     >>> getpath({'a': {}}, ['a', 'b']) is NOTHING
356     True
357     """
358     for k in ks:
359         if not isinstance(t, dict) or k not in t:
360             return default
361         t = t[k]
362     return t
363
364 # multiset ops on count-dicts (use .get -- indexing a Counter autovivifies):
365 bag_diff = lambda a, b: {k: a[k] - b.get(k, 0) for k in a if a[k] > b.get(k, 0)}
366 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a[k], b[k]))}
367 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
368 bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items())
369
370 # ===== 13. Maybe ===== (NOTHING sentinel; None is a legal value)
371 def bind(x, f):
372     """>= : short-circuit on NOTHING.
373     >>> bind(3, lambda v: v + 1), bind(NOTHING, lambda v: v + 1)
374     (4, NOTHING)
375     """
376     return NOTHING if x is NOTHING else f(x)
377
378 isJust = lambda x: x is not NOTHING
379 isNothing = lambda x: x is NOTHING
380 fromMaybe = lambda default, x: default if x is NOTHING else x
381
382 listToMaybe = lambda xs: xs[0] if xs else NOTHING
383 maybeToList = lambda x: [] if x is NOTHING else [x]
384
385 def catMaybes(xs):
386     """
387     >>> catMaybes([1, NOTHING, None, 2])
388     [1, None, 2]
389     """
390     return [x for x in xs if x is not NOTHING]
391
392 def mapMaybe(f, xs):
393     """Map into Maybe and drop the NOTHINGS, one pass.
394     >>> mapMaybe(lambda s: int(s) if s.isdigit() else NOTHING, ['3', 'x', '7'])
395     [3, 7]
396     """
397     return [y for y in map(f, xs) if y is not NOTHING]
398
399 def sequenceM(xs):
400     """All-or-nothing.
401     >>> sequenceM([1, 2]), sequenceM([1, NOTHING])
402     ([1, 2], NOTHING)
403     """
404     out = []
405     for x in xs:
406         if x is NOTHING:
407             return NOTHING
408         out.append(x)
409     return out
410
411 traverseM = lambda f, xs: sequenceM(map_(f, xs))
412
413 def maybe_get(d, k):
414     """dict lookup into Maybe (None stays distinguishable from missing).
415     >>> maybe_get({'a': None}, 'a'), maybe_get({}, 'a')
416     (None, NOTHING)
417     """
418     return d.get(k, NOTHING)
419
420 # ===== 14. Either ===== (tagged tuples; match-friendly)
421 Ok = lambda v: ("ok", v)
422 Err = lambda e: ("err", e)
423
424 def bindE(x, f):

```

```

425     """
426     >>> bindE(Ok(3), lambda v: Ok(v + 1)), bindE(Err("no"), lambda v: Ok(v))
427     (('ok', 4), ('err', 'no'))
428     """
429     return x if x[0] == "err" else f(x[1])
430
431 def sequenceE(xs):
432     """First Err wins; else Ok of all values.
433     >>> sequenceE([Ok(1), Err("a"), Err("b")])
434     ('err', 'a')
435     """
436     out = []
437     for x in xs:
438         if x[0] == "err":
439             return x
440         out.append(x[1])
441     return Ok(out)
442
443 def traverseE(f, xs):
444     """Map into Either, then sequence.
445     >>> traverseE(lambda n: Ok(n * 2) if n else Err('zero'), [1, 2])
446     ('ok', [2, 4])
447     """
448     return sequenceE(map_(f, xs))
449
450 def partitionEithers(xs):
451     """
452     >>> partitionEithers([Ok(1), Err('a'), Ok(2)])
453     ([1, 2], ['a'])
454     """
455     oks, errs = [], []
456     for tag, v in xs:
457         (oks if tag == "ok" else errs).append(v)
458     return (oks, errs)
459
460 def note(msg, x):
461     """Maybe -> Either: attach WHY to a NOTHING.
462     >>> note("missing key", NOTHING)
463     ('err', 'missing key')
464     """
465     return Err(msg) if x is NOTHING else Ok(x)
466
467 # ===== 15. do-notation ===== (yield is >>=; linear monads only)
468 def do(binder, is_fail):
469     """Build a do-block decorator for any short-circuiting monad."""
470     def deco(gen_fn):
471         def run(*a, **kw):
472             g = gen_fn(*a, **kw)
473             def step(val):
474                 try:
475                     m = g.send(val)
476                     except StopIteration as e:
477                         return e.value
478                     return binder(m, step)
479             return step(None)
480         return run
481     return deco
482
483 doM = do(bind, lambda x: x is NOTHING) # generator returns a plain value
484 doE = do(bindE, lambda x: x[0] == "err") # generator returns Ok(...)/Err(...)
485
486 # ===== 16. parser combinators =====
487 # Parser a = str -> (value, rest) | FAIL. Values may be None (json null!).
488 def doP(gen_fn):
489     """do-notation that THREADS the input: yield a parser, receive its value.
490     Fresh generator per parse -- re-runnable, backtrack-safe under alt."""
491     def make(*a, **kw):
492         def parser(s):
493             g, val = gen_fn(*a, **kw), None
494             try:
495                 while True:
496                     r = g.send(val)(s)
497                     if r is FAIL:
498                         return FAIL

```

```

499         val, s = r
500     except StopIteration as e:
501         return (e.value, s)
502     return parser
503     return make
504
505 def alt(*ps):
506     """</> : first success wins."""
507     return lambda s: next((r for r in (p(s) for p in ps) if r is not FAIL), FAIL)
508
509 def many(p):
510     """Zero or more, loop not recursion; stalls guard against empty matches."""
511     def parser(s):
512         out = []
513         while (r := p(s)) is not FAIL and r[1] != s:
514             out.append(r[0]); s = r[1]
515         return (out, s)
516     return parser
517
518 def many1(p):
519     def parser(s):
520         r = p(s)
521         if r is FAIL:
522             return FAIL
523         rest = many(p)(r[1])
524         return ([r[0]] + rest[0], rest[1])
525     return parser
526
527 def sepBy(p, sep):
528     def pair(s):
529         r = sep(s)
530         return FAIL if r is FAIL else p(r[1])
531     def parser(s):
532         r = p(s)
533         if r is FAIL:
534             return ([], s)
535         rest = many(pair)(r[1])
536         return ([r[0]] + rest[0], rest[1])
537     return parser
538
539 def lit(c):
540     """One expected character, whitespace-blind."""
541     def parser(s):
542         s = s.lstrip()
543         return (c, s[1:]) if s[:1] == c else FAIL
544     return parser
545
546 def rx(pat, conv=identity):
547     """Regex token, whitespace-blind, converted."""
548     r = _re.compile(pat)
549     def parser(s):
550         s = s.lstrip()
551         m = r.match(s)
552         return (conv(m.group()), s[m.end():]) if m else FAIL
553     return parser
554
555 def chainl1(p, ops):
556     """p (op p)* folded LEFT; ops maps operator char -> binary function."""
557     opp = alt(*[lit(c) for c in ops])
558     def parser(s):
559         vr = p(s)
560         while vr is not FAIL:
561             op = opp(vr[1])
562             if op is FAIL:
563                 return vr
564             w = p(op[1])
565             if w is FAIL:
566                 return vr
567             vr = (ops[op[0]](vr[0], w[0]), w[1])
568         return FAIL
569     return parser
570
571 def runParser(p, s):
572     """Parser -> Either: Ok(value) or Err showing where it stopped.

```

```

573 >>> runParser(rx(r'\d+', int), "42")
574 ('ok', 42)
575 >>> runParser(rx(r'\d+', int), "42x")
576 ('err', "unconsumed input: 'x'")
577 """
578 r = p(s)
579 if r is FAIL:
580     return Err(f"no parse: {s!r}")
581 if r[1].strip():
582     return Err(f"unconsumed input: {r[1].strip()!r}")
583 return Ok(r[0])
584
585 # ===== 17. monoids =====
586 Monoid = lambda empty, op: (empty, op)
587 mconcat = lambda m, xs: foldl(m[1], xs, m[0])
588 foldMap = lambda f, m, xs: mconcat(m, [f(x) for x in xs])
589 both = lambda m1, m2: Monoid((m1[0], m2[0]),
590                               lambda a, b: (m1[1](a[0], b[0]), m2[1](a[1], b[1]))))
591 Sum = Monoid(0, add)
592 Product = Monoid(1, mul)
593 All = Monoid(True, lambda a, b: a and b)
594 Any = Monoid(False, lambda a, b: a or b)
595 MaxM = Monoid(float("-inf"), max)
596 MinM = Monoid(float("inf"), min)
597 ListM = Monoid([], add)
598 First = Monoid(NOTHING, lambda a, b: b if a is NOTHING else a)
599 Last = Monoid(NOTHING, lambda a, b: a if b is NOTHING else b)
600
601 if __name__ == "__main__":
602     import doctest
603     fails, total = doctest.testmod()
604     print(f"{total - fails}/{total} doctests passed" if not fails
605           else f"{fails} FAILED")

```

